

ABDUL WALI KHAN UNIVERSITY MARDAN

Department of Computer Science

BS Final Year Project Thesis

BSCS (Software Engineering)

**DESIGN AND IMPLEMENTATION OF SMART PERSONAL
TASK MANAGEMENT SYSTEM**

A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree of

Bachelor of Science in Computer Science (Software Engineering)

Department of Computer Science
Abdul Wali Khan University Mardan

Submitted By:

Farhan Khan (AWKUM-22147646)

Kamran Khan (AWKUM-22157075)

Muneeb Diyar Khan (AWKUM-22147942)

Supervisor:

Dr. Shahzad

Year: 2025–2026

DEDICATION

We dedicate this work to our parents and families,
whose unwavering support, prayers, and encouragement
have been the foundation of everything we have achieved.

We also dedicate this work to our supervisor, Dr. Shahzad,
for his guidance and mentorship throughout this journey.

DECLARATION

We hereby declare that the work presented in this thesis, titled "Design and Implementation of Smart Personal Task Management System," is our own original work and has not been submitted previously, either in whole or in part, for any degree or qualification at this or any other institution. All sources of information and references used in this thesis have been properly acknowledged and cited in accordance with academic standards.

We further declare that this thesis does not contain any material previously published or written by another person, except where due reference is made in the text. The work presented herein was carried out under the supervision of Dr. Shahzad, Department of Computer Science, Abdul Wali Khan University Mardan.

Student Name: Farhan Khan

Registration No.: AWKUM-22147646

Signature: _____

Date: _____

Student Name: Kamran Khan

Registration No.: AWKUM-22157075

Signature: _____

Date: _____

Student Name: Muneeb Diyar Khan

Registration No.: AWKUM-22147942

Signature: _____

Date: _____

CERTIFICATE OF APPROVAL

This is to certify that the Final Year Project thesis titled "Design and Implementation of Smart Personal Task Management System" has been carried out by Farhan Khan (AWKUM-22147646), Kamran Khan (AWKUM-22157075), and Muneeb Diyar Khan (AWKUM-22147942) under my supervision. The work presented in this thesis meets the required standards and is approved for submission in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science (Software Engineering) from Abdul Wali Khan University Mardan.

To the best of my knowledge, the work presented herein is original and has not been submitted elsewhere for any other degree or award. The project demonstrates a satisfactory level of technical competence and academic rigor required for the award of the said degree.

Supervisor:

Name: Dr. Shahzad

Designation: Lecturer / Assistant Professor

Department: Computer Science

University: Abdul Wali Khan University Mardan

Signature: _____

Date: _____

Head of Department:

Name: _____

Signature: _____

Date: _____

ABSTRACT

In the modern era of digital productivity, effective task management has become an essential requirement for students, professionals, and individuals who seek to organize their daily responsibilities efficiently. This thesis presents the design and implementation of Task Flow — a Smart Personal Task Management System — a full-stack web application developed to assist individuals in managing their tasks, tracking productivity, and maintaining organized workflows through an intuitive and responsive interface.

The system was developed using a modern technology stack consisting of HTML5, CSS3, and vanilla JavaScript for the frontend, while the backend is powered by Node.js with the Express.js framework (version 5). The application uses PostgreSQL as its relational database management system for persistent data storage. User authentication is handled through session-based mechanisms using the express-session library, with passwords securely hashed using bcryptjs. The application was initially targeted for deployment on the Railway cloud platform; however, due to platform constraints encountered during the testing phase, it currently operates successfully in a local development environment.

Task Flow provides users with a comprehensive set of features including secure user registration and authentication, task creation with title, description, due date, priority level (high, medium, low), and category assignment. Users can view and manage tasks across three status states — pending, completed, and overdue. The system includes a dedicated statistics page offering visual analytics through bar charts and doughnut charts powered by Chart.js, displaying priority distributions and task completion rates. Custom category management allows users to organize their tasks under color-coded labels. The application is fully responsive, supporting both desktop and mobile devices.

The development methodology followed was Agile, allowing iterative development and continuous improvement across multiple sprints. The system was designed with a client-server architecture where the frontend communicates with the backend through a RESTful API. Security measures including input validation, rate limiting, and XSS protection have been implemented throughout the application.

The system was tested locally and all core functionalities were verified to work correctly. The thesis concludes with a discussion of limitations, including the absence of mobile application support and real-time collaborative features, and proposes future enhancements such as mobile app development, AI-based task suggestions, and real-time collaboration features.

Keywords: Task Management, Web Application, Node.js, Express.js, PostgreSQL, RESTful API, Agile Development, Productivity System, JavaScript, Full-Stack Development.

ACKNOWLEDGEMENTS

First and foremost, we are deeply grateful to Almighty Allah for granting us the knowledge, patience, and strength to complete this Final Year Project. Without His blessings, this work would not have been possible.

We would like to express our sincere gratitude and appreciation to our supervisor, Dr. Shahzad, for his invaluable guidance, continuous support, and constructive feedback throughout the development of this project. His encouragement and technical insights were instrumental in shaping the direction of this work.

We also extend our heartfelt thanks to the faculty and staff of the Department of Computer Science, Abdul Wali Khan University Mardan, for providing us with a solid foundation of knowledge and an environment conducive to academic growth. The coursework and practical exposure we received during our four years of study equipped us with the skills necessary to undertake this project.

Special thanks go to our families for their unwavering love, encouragement, and sacrifices. Their moral support and prayers kept us motivated throughout the challenging phases of this project.

Finally, we acknowledge the open-source community and the developers behind the frameworks, libraries, and tools we used in this project — including Node.js, Express.js, PostgreSQL, Chart.js, and Font Awesome — whose contributions made our work significantly more achievable.

Farhan Khan

Kamran Khan

Muneeb Diyar Khan

Department of Computer Science

Abdul Wali Khan University Mardan

2025–2026

TABLE OF CONTENTS

Dedication	i
Declaration	ii
Certificate of Approval	iii
Abstract	iv
Acknowledgements	v
Table of Contents	vi
List of Figures	vii
List of Tables	viii
Chapter 1: Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives of the Study	4
1.4 Scope of the Project	5
1.5 Significance of the Study	6
1.6 Structure of the Thesis	7
Chapter 2: Literature Review	8
2.1 Overview of Task Management Systems	8
2.2 Existing Tools and Comparative Analysis	9
2.3 Web Technologies for Task Management	11
2.4 User Authentication and Security	13
2.5 Data Visualization in Productivity Tools	14

2.6 Research Gaps and Justification	15
Chapter 3: Methodology	16
3.1 Software Development Methodology	16
3.2 Agile Development Process	17
3.3 Requirements Engineering	18
3.4 Technology Stack Selection	20
3.5 Development Tools and Environment	21
Chapter 4: System Design	22
4.1 System Architecture	22
4.2 Database Design	24
4.3 API Design	26
4.4 User Interface Design	27
4.5 Security Design	29
Chapter 5: Implementation	30
5.1 Frontend Implementation	30
5.2 Backend Implementation	33
5.3 Database Implementation	36
5.4 Authentication Module	37
5.5 Task Management Module	38
5.6 Statistics and Analytics Module	39
Chapter 6: Results and Discussion	40
6.1 System Testing	40
6.2 Functional Testing Results	41
6.3 User Interface Screenshots	43

6.4 Performance Analysis	45
6.5 Discussion	46
Chapter 7: Conclusion and Future Work	47
7.1 Conclusion	47
7.2 Limitations	48
7.3 Future Work	49
References	50
Appendices	52

LIST OF FIGURES

Figure 1.1: Overview of Task Flow System Architecture	23
Figure 1.2: Login Page of Task Flow	28
Figure 1.3: Sign Up / Registration Page	28
Figure 1.4: Dashboard – Main Task View	29
Figure 1.5: Add New Task Modal	29
Figure 1.6: Statistics Page – Summary Cards	43
Figure 1.7: Statistics Page – Priority Breakdown Table	44
Figure 1.8: Entity Relationship Diagram (ERD)	25
Figure 1.9: Client-Server Architecture Diagram	23
Figure 1.10: Agile Sprint Cycle Diagram	17

LIST OF TABLES

Table 2.1: Comparison of Existing Task Management Systems	10
Table 3.1: Functional Requirements	19
Table 3.2: Non-Functional Requirements	20
Table 3.3: Technology Stack Summary	21
Table 4.1: Database Tables Description	24
Table 4.2: API Endpoints Summary	26
Table 6.1: Functional Test Cases and Results	42

CHAPTER 1

Introduction

1.1 Background and Motivation

In today's fast-paced world, individuals across all walks of life — students, professionals, entrepreneurs, and academics — are continuously faced with the challenge of managing multiple responsibilities simultaneously. The ability to organize tasks effectively, track deadlines, prioritize work, and monitor progress has become not merely a desirable skill but an essential one for personal and professional success. With the proliferation of digital devices and internet connectivity, software-based task management systems have emerged as the most viable and convenient solution to this challenge.

Task management refers to the process of managing a task through its lifecycle, from planning and tracking to analysis and reporting. Effective task management encompasses setting priorities, assigning deadlines, categorizing work, and reviewing completion rates. Historically, individuals relied on physical planners, sticky notes, and paper-based calendars to manage their tasks. While these analog methods served their purpose for decades, they suffered from obvious limitations: lack of accessibility, inability to provide analytics, no support for reminders, and difficulty scaling as task volumes grew.

The digital revolution brought with it a wave of task management applications that addressed many of these limitations. Tools such as Todoist, Trello, Microsoft To-Do, Asana, and Notion gained widespread adoption by offering cloud-based, feature-rich environments for task and project management. However, many of these tools are either excessively complex for individual personal use, require expensive subscriptions, rely heavily on third-party cloud services, or impose feature limitations on free-tier users. There exists a clear need for a lightweight, clean, and fully functional personal task management system that can be deployed and used without dependency on external platforms or premium subscriptions.

This project was motivated by the recognition of this gap and the desire to build a practical, academically rigorous, and technically sound system that addresses the core needs of individual task management. The system — named Task Flow — was conceived as a full-stack web application that provides all the essential features of a personal productivity tool, including task creation and management, priority assignment, category organization, real-time

status tracking, and visual analytics. The project was undertaken as the Final Year Project by three students of the Department of Computer Science at Abdul Wali Khan University Mardan, combining theoretical knowledge and practical software engineering skills acquired throughout the degree program.

1.2 Problem Statement

Despite the wide availability of task management tools, a significant portion of individuals — particularly students and early-career professionals in developing regions — lack access to suitable, affordable, and easy-to-use task management software. The existing solutions present several practical challenges:

- Many popular task management platforms are subscription-based and their advanced features are locked behind paywalls, making them inaccessible to users with limited financial resources.
- Several existing tools are feature-heavy and designed for team collaboration rather than individual personal use, resulting in steep learning curves and cluttered interfaces that overwhelm individual users.
- Cloud-dependent solutions require constant internet connectivity and raise concerns regarding data privacy, as user data is stored on remote third-party servers.
- Most freely available web-based tools do not offer meaningful productivity analytics or visual statistics that help users understand their task completion patterns and prioritization habits.
- There is a lack of lightweight systems that can be self-hosted or run locally with minimal infrastructure requirements, which is particularly relevant in regions with inconsistent internet access.

These challenges collectively create a barrier for individuals seeking simple yet effective personal task management. The proposed system, Task Flow, addresses each of these pain points by providing a self-hostable, locally deployable, feature-complete personal task management web application built on open-source technologies. The system is designed to be intuitive, efficient, and visually appealing while keeping the user experience focused on individual productivity rather than team workflows.

1.3 Objectives of the Study

The primary objective of this project is to design and implement a smart personal task management system that enables users to efficiently create, organize, track, and analyze their personal tasks through a clean and responsive web interface. The specific objectives of the project are as follows:

1. To design and develop a secure, session-based user authentication system that allows individuals to register, log in, and manage their own private task data with appropriate security measures including password hashing and input validation.
2. To implement a comprehensive task management module that supports task creation with attributes including title, description, due date, priority level (high, medium, or low), and category assignment, along with full CRUD (Create, Read, Update, Delete) operations on tasks.
3. To develop a category management system that allows users to create custom, color-coded categories for organizing tasks according to their personal workflow and organizational needs.
4. To build a real-time task status tracking mechanism that differentiates between pending, completed, and overdue tasks, allowing users to mark tasks as complete, restore completed tasks to pending, and filter tasks based on priority and category.
5. To implement an interactive statistics and analytics module that provides visual representations of task data including bar charts for priority distribution and doughnut charts for task status, alongside a tabular priority breakdown with completion percentages.
6. To develop a fully responsive user interface that functions seamlessly across desktop and mobile devices, ensuring accessibility without additional platform-specific development.
7. To design and implement a RESTful API backend that facilitates clean communication between the frontend and the PostgreSQL database, following best practices for API security, error handling, and data validation.

8. To apply the Agile software development methodology throughout the project lifecycle, ensuring iterative development, continuous testing, and incremental delivery of features.

1.4 Scope of the Project

The scope of this project encompasses the complete design, development, testing, and documentation of a web-based personal task management system intended for individual use. The following defines the boundaries of the system:

The system is a single-user-per-account application. Each registered user has access exclusively to their own tasks and categories, ensuring data isolation and privacy. The application does not support multi-user collaboration, shared task lists, or team-based project management within the current version.

The frontend of the system is developed using standard web technologies — HTML5, CSS3, and vanilla JavaScript — without dependency on any frontend framework such as React, Angular, or Vue. This design decision was made intentionally to maintain simplicity, minimize dependencies, and demonstrate proficiency in core web development fundamentals. The backend is developed using Node.js with the Express.js framework (version 5), and data is persisted using a PostgreSQL relational database.

The system is designed to run in a local development environment accessible through a standard web browser. While deployment to the Railway cloud platform was initially planned, the current version operates as a locally hosted application accessible via <http://localhost:3000>. This scope limitation does not affect the functionality or completeness of the system.

Features explicitly outside the scope of the current version include: mobile native application development, email-based notification systems, calendar integration, real-time multi-user collaboration, and AI-based task recommendation engines. These features are identified as potential areas for future development and are discussed in Chapter 7.

1.5 Significance of the Study

This project holds significance on multiple levels — academic, technical, and practical. From an academic perspective, it demonstrates the integration of multiple disciplines of

computer science, including software engineering, database management, web development, user interface design, and systems security, into a single coherent applied project. It serves as evidence of the students' ability to translate theoretical knowledge into a functional software product.

From a technical perspective, the project showcases the design and implementation of a full-stack web application using a modern and industry-relevant technology stack. The use of Node.js, Express.js, and PostgreSQL reflects current industry standards for backend web development. The implementation of session-based authentication, RESTful API design, rate limiting, XSS protection, and input sanitization demonstrates an awareness of real-world security concerns in web applications.

From a practical perspective, the system provides genuine utility to its users. Task Flow offers a clean, distraction-free environment for managing personal tasks without the complexity or cost associated with enterprise-grade project management tools. It is particularly suited for students, freelancers, and individuals who need a simple yet powerful tool to stay organized.

Furthermore, the open-source nature of the technology stack used in this project means that the system can be extended, modified, and redistributed freely. The codebase can serve as a learning resource for future students and developers interested in full-stack web development with Node.js and PostgreSQL.

1.6 Structure of the Thesis

This thesis is organized into seven chapters, each covering a distinct aspect of the project. The structure is as follows:

Chapter 1 — Introduction: Provides the background, motivation, problem statement, objectives, scope, and significance of the project. It establishes the context and justification for the work undertaken.

Chapter 2 — Literature Review: Reviews existing task management systems and related research, analyzes comparable tools, discusses relevant web technologies, and identifies the research gap that this project addresses.

Chapter 3 — Methodology: Describes the Agile software development methodology adopted for this project, outlines the sprint-based development process, and details the requirements engineering process including functional and non-functional requirements.

Chapter 4 — System Design: Presents the architectural design of the system, including the client-server architecture, database schema, Entity-Relationship Diagram (ERD), API endpoint design, and user interface wireframes.

Chapter 5 — Implementation: Describes the actual coding and development process, covering the frontend implementation, backend modules, database setup, authentication system, task management module, and the statistics and analytics engine.

Chapter 6 — Results and Discussion: Presents the outcomes of functional testing, discusses the system's performance, provides annotated screenshots of key system interfaces, and evaluates the extent to which the project objectives were met.

Chapter 7 — Conclusion and Future Work: Summarizes the key contributions of the project, acknowledges the current limitations of the system, and proposes directions for future enhancement and research.

CHAPTER 2

Literature Review

2.1 Overview of Task Management Systems

Task management systems are software applications designed to help individuals or teams create, organize, assign, track, and complete tasks within a given timeframe. The concept of digital task management evolved from simple to-do list applications in the early days of personal computing to sophisticated, cloud-based project management platforms that are now integral to modern organizational workflows. The academic and industrial interest in task management software has grown significantly over the past two decades, driven by increasing demands for personal and organizational productivity in an ever-accelerating professional environment.

Early research in this area focused primarily on human-computer interaction and user interface design for productivity tools. Studies by Czerwinski et al. (2004) on task management in information workers highlighted the cognitive challenges individuals face when managing multiple concurrent tasks and interruptions, establishing a theoretical foundation for the design of effective task management interfaces. Their findings indicated that users benefit significantly from visual organization, priority indicators, and contextual grouping of tasks — insights that directly informed the design decisions made in the Task Flow system.

The emergence of web-based applications in the mid-2000s marked a turning point in task management software development. Browser-accessible tools eliminated the need for software installation and enabled cross-device synchronization, dramatically increasing adoption rates. Subsequent research by Bellotti et al. (2004) examined the role of email in task management and found that a significant proportion of knowledge workers use their email inbox as a de facto task list, highlighting the absence of suitable dedicated task management tools and the consequent need for purpose-built systems.

More recent literature has shifted toward examining the psychological and behavioral dimensions of task management software usage. Studies have shown that features such as due date reminders, priority labeling, and progress visualization contribute positively to user motivation and task completion rates. A study by Macan (1994) on time management training established that individuals who employ systematic task prioritization and scheduling

demonstrate significantly higher productivity levels compared to those without structured task management practices. These behavioral insights reinforced the importance of building systems that not only organize tasks but actively support users in understanding their productivity patterns through analytics.

2.2 Existing Tools and Comparative Analysis

A review of existing task management tools reveals a diverse landscape of applications catering to different user groups and use cases. This section examines several prominent tools and evaluates them against the requirements that motivated the development of Task Flow.

Todoist is one of the most widely used personal task management applications globally. It offers a clean interface, natural language input for task creation, priority levels, labels, and project-based organization. However, its advanced features — including reminders, filters, and productivity reports — are restricted to the premium subscription tier. Furthermore, all data is stored on Todoist's proprietary servers, raising data privacy concerns for users who prefer local or self-hosted solutions.

Microsoft To-Do, formerly known as Wunderlist, is a free task management application integrated with the Microsoft 365 ecosystem. It provides a simple and clean interface with support for task lists, due dates, and reminders. However, its functionality is relatively limited compared to other tools in the category, and its tight integration with the Microsoft ecosystem means that users outside this ecosystem experience a degraded experience. It does not provide statistical analytics or data visualization features.

Trello is a visual project management tool based on the Kanban board methodology. While highly effective for team-based project management, Trello's board-centric approach is not ideally suited for individual personal task management, where a simple task list with priority and category filtering is more appropriate. Its free tier restricts the number of power-ups (integrations) available, and the interface can feel unnecessarily complex for simple personal use cases.

Asana is a comprehensive project and task management platform designed primarily for team collaboration. It offers extensive features including task assignments, project timelines, workload management, and integration with hundreds of third-party applications. However, its

complexity and pricing make it poorly suited for individual use. The learning curve is steep for users who simply want to manage personal daily tasks.

Notion is a highly flexible all-in-one workspace that combines note-taking, database management, and task tracking. While its flexibility is a significant advantage, it also represents a major drawback for users seeking a focused, purpose-built task management experience. The setup required to create an effective task management system within Notion is non-trivial, and the platform's general-purpose nature can be distracting for users with simple task management needs.

Table 2.1: Comparison of Existing Task Management Systems

Feature	Todoist	MS To-Do	Trello	Asana	Notion	Task Flow
Free to Use	Partial	Yes	Partial	Partial	Partial	Yes
Priority Levels	Yes	No	No	Yes	Custom	Yes
Categories/Labels	Yes	Limited	Labels	Yes	Custom	Yes
Analytics/Stats	Premium	No	No	Premium	Manual	Yes (Built-in)
Self-Hostable	No	No	No	No	No	Yes
Individual Focus	Yes	Yes	No	No	Partial	Yes
Open Source Stack	No	No	No	No	No	Yes
Overdue Detection	Yes	Yes	No	Yes	Manual	Yes

The comparative analysis presented in Table 2.1 reveals that Task Flow provides a unique combination of features that no single existing tool offers in a free, self-hostable package. The inclusion of built-in analytics, priority management, custom categories, and overdue detection — all within an open-source, individually focused system — positions Task Flow as a distinctive contribution to the personal productivity software landscape.

2.3 Web Technologies for Task Management

The choice of web technologies plays a critical role in determining the performance, scalability, maintainability, and security of a web-based task management system. This section

reviews the key technologies employed in the development of Task Flow and situates them within the broader context of modern web development practices.

Node.js, introduced by Ryan Dahl in 2009, is a server-side JavaScript runtime built on Google Chrome's V8 engine. Its non-blocking, event-driven I/O model makes it particularly well-suited for building scalable network applications. Studies comparing Node.js with traditional server-side technologies such as PHP and Java have demonstrated that Node.js provides superior performance for I/O-intensive applications with concurrent users, making it an appropriate choice for a web application involving frequent database reads and writes. The use of JavaScript on both the frontend and backend also reduces the cognitive overhead for developers and facilitates code reuse.

Express.js is the most widely used web application framework for Node.js. Its minimalist design philosophy provides developers with a thin layer of abstraction over the Node.js HTTP module, offering routing, middleware support, and request/response handling without imposing a rigid application structure. Express.js version 5, used in this project, introduced improvements to error handling and asynchronous route support. The framework's simplicity and flexibility make it particularly suitable for building RESTful APIs, which are the backbone of the Task Flow backend.

PostgreSQL is a powerful, open-source object-relational database management system (ORDBMS) with over 35 years of active development. It is known for its robustness, standards compliance, extensibility, and support for advanced data types. For the Task Flow system, PostgreSQL provides the persistent storage layer for user accounts, tasks, and categories. The use of UUID primary keys, foreign key constraints, check constraints for enumerated fields (priority, status), and cascading deletes ensures data integrity and relational consistency throughout the application.

The frontend of Task Flow is built using vanilla HTML5, CSS3, and JavaScript without any frontend framework. This decision aligns with a growing trend in web development toward reducing dependency on heavy JavaScript frameworks for applications that do not require the complexity they introduce. The use of CSS custom properties (variables) for theming, Flexbox and CSS Grid for layout, and the Fetch API for asynchronous communication with the backend reflects a modern and standards-compliant approach to frontend development. The Plus Jakarta

Sans font, loaded from Google Fonts, and Font Awesome icon library contribute to the professional and polished visual appearance of the application.

2.4 User Authentication and Security in Web Applications

Security is a foundational concern in any web application that handles user data. The literature on web application security identifies authentication, authorization, input validation, and protection against common attack vectors as the primary areas requiring careful design attention.

Session-based authentication, as implemented in Task Flow using the express-session library, represents a well-established approach to managing user identity in web applications. In this model, the server creates a session upon successful login and stores a session identifier in a cookie on the client side. Subsequent requests from the client include this cookie, allowing the server to identify the authenticated user. While token-based authentication using JSON Web Tokens (JWT) has gained considerable popularity in recent years — particularly for stateless, scalable APIs — session-based authentication remains a practical and secure choice for monolithic web applications where the frontend and backend are served from the same origin, as is the case with Task Flow.

Password security is addressed through the use of bcryptjs, a JavaScript implementation of the bcrypt password hashing function. Bcrypt is widely recommended for password storage due to its deliberate computational cost, which makes brute-force attacks significantly more expensive compared to faster hashing algorithms such as MD5 or SHA-1. The use of a salt factor of 10 in the Task Flow implementation provides a strong balance between security and performance.

Cross-Site Scripting (XSS) protection is implemented through the escapeHtml() utility function in the frontend JavaScript, which sanitizes all user-generated content before it is inserted into the DOM. This prevents malicious scripts injected through task titles or descriptions from executing in the user's browser. Rate limiting, implemented using the express-rate-limit middleware, protects the authentication endpoints from brute-force and credential-stuffing attacks by restricting the number of requests a client can make within a configurable time window.

2.5 Data Visualization in Productivity Applications

Data visualization has become an increasingly important component of productivity software. Research in human-computer interaction has consistently shown that visual representations of data — charts, graphs, and progress indicators — are more effective than raw numerical data for helping users quickly understand patterns, trends, and performance metrics. For task management systems, visualization serves to make the abstract concept of "productivity" tangible and actionable.

Chart.js is a widely adopted open-source JavaScript charting library that provides a clean, responsive, and highly customizable charting API. It supports a diverse range of chart types including bar charts, line charts, doughnut charts, radar charts, and more. For Task Flow, Chart.js version 4.4.1 is used to render two key visualizations on the statistics page: a bar chart displaying the distribution of tasks across priority levels (high, medium, low), and a doughnut chart showing the proportion of completed versus pending tasks. These visualizations are rendered on HTML5 canvas elements and are fully responsive, adapting their dimensions to the available screen space.

Beyond charts, the statistics page features a priority breakdown table that displays, for each priority level, the total number of tasks, the number completed, the number pending, and the completion percentage accompanied by a color-coded progress bar. This multi-faceted approach to productivity visualization ensures that users receive a comprehensive picture of their task management behavior, enabling them to identify patterns such as a tendency to create high-priority tasks but complete a disproportionately lower percentage of them — a common productivity anti-pattern identified in behavioral productivity research.

2.6 Research Gaps and Justification

The review of existing literature and tools reveals several gaps that this project seeks to address. First, while there is extensive research on enterprise-level project management software and team collaboration tools, relatively little attention has been paid to the design of lightweight, self-hostable personal task management systems specifically tailored to the needs of individual users in resource-constrained environments.

Second, the majority of commercially available task management tools restrict their analytics and visualization features to paid tiers, creating an inequality of access that disadvantages students and users in developing countries. Task Flow addresses this gap by integrating a full-featured analytics module into a completely free and open-source system.

Third, existing academic projects in the task management domain — particularly those produced as university final year projects — often focus narrowly on either the frontend or the backend without delivering a cohesive, production-quality full-stack application. This project distinguishes itself by delivering a complete system with a polished, responsive user interface, a secure and well-structured backend, a relational database with proper schema design, and comprehensive documentation.

Fourth, the choice to implement the project without a frontend framework represents a deliberate pedagogical decision. By relying exclusively on vanilla JavaScript, the project demonstrates a deep understanding of browser APIs, the Document Object Model (DOM), asynchronous programming, and event handling — foundational skills that are often obscured by the abstraction layers of modern JavaScript frameworks.

In summary, the literature review validates the need for the Task Flow system and confirms that the approach taken — combining modern full-stack web technologies with a focus on individual productivity, built-in analytics, and security best practices — represents a meaningful and original contribution to the domain of personal task management software.

CHAPTER 3

Methodology

3.1 Introduction to Software Development Methodology

A software development methodology is a structured framework that defines the processes, practices, roles, and tools used to plan, develop, test, and deliver software. The selection of an appropriate methodology is one of the most critical decisions in any software project, as it fundamentally shapes how the development team approaches problem-solving, manages change, handles uncertainty, and delivers value incrementally to stakeholders. Different methodologies are suited to different project contexts — a methodology appropriate for a large-scale enterprise system with a fixed, fully documented set of requirements may be entirely unsuitable for a smaller, exploratory student-led project where requirements evolve through iterative prototyping and continuous reflection.

For the development of Task Flow, the project team carefully evaluated three mainstream software development methodologies before selecting the most appropriate framework. The methodologies considered were the Waterfall model, the Spiral model, and the Agile methodology. Each is described below, followed by a detailed justification for the final selection.

The Waterfall model is a sequential, plan-driven methodology in which each phase of the software development lifecycle — requirements analysis, system design, implementation, testing, deployment, and maintenance — must be completed in full before the subsequent phase commences. The Waterfall model offers clarity, predictability, and a straightforward documentation trail, making it suitable for projects where all requirements are stable and completely understood at the outset. However, for Task Flow, several design and feature decisions were made iteratively based on practical prototyping and testing during development. The rigid sequential nature of Waterfall would have prevented the team from incorporating these mid-development insights without costly and disruptive phase reversals, making it an unsuitable choice for this project.

The Spiral model combines elements of the Waterfall model with iterative development and places particular emphasis on formal risk identification and risk mitigation at each cycle. While theoretically robust, the Spiral model introduces a level of formal overhead —

especially in its structured risk assessment procedures — that is disproportionate for a three-member student project with a development timeline of approximately twelve months and a relatively contained set of known risks.

The Agile methodology, by contrast, is an iterative and incremental approach to software development that prioritizes adaptability, close collaboration, working software, and responsiveness to evolving requirements. Agile development proceeds through a series of short, time-boxed development cycles called sprints, each of which produces a potentially shippable increment of functional software. This approach aligns naturally with the exploratory character of this project, where new features were identified, prototyped, tested, and refined throughout the development lifecycle rather than being fully prescribed in an upfront requirements specification document.

3.2 Agile Development Methodology

Agile is not a single prescriptive methodology but rather an umbrella term for a family of iterative development approaches that share a common set of values and principles as codified in the Agile Manifesto, first published in 2001 by seventeen experienced software practitioners. The four core values of the Agile Manifesto are: (1) individuals and interactions over processes and tools; (2) working software over comprehensive documentation; (3) customer collaboration over contract negotiation; and (4) responding to change over following a rigid plan. These values directly informed the project team's approach to the development of Task Flow at every stage.

The specific Agile framework adopted draws primarily from Scrum, which organizes work into fixed-length sprints, each beginning with a sprint planning session and concluding with a sprint review and retrospective. While the full Scrum framework defines specific roles — Product Owner, Scrum Master, and Development Team — and formal ceremonies including daily standups, sprint reviews, and retrospectives, a scaled-down adaptation of these practices was employed to suit the three-member team structure and academic project context.

3.2.1 Agile Principles Applied in Task Flow

The following Agile principles were explicitly observed and applied throughout the development of Task Flow:

- **Iterative Development:** The system was built in multiple sequential sprints, with each sprint delivering a functional, testable software increment. The team did not attempt to build the complete system in a single continuous effort but instead constructed and validated features incrementally, beginning with the most fundamental components — server setup, database schema, and user authentication — before progressing to task management, category handling, and finally analytics and UI polish.
- **Continuous Testing and Integration:** Testing was performed continuously throughout development rather than being deferred to a dedicated post-implementation testing phase. Each new feature or module was verified against its requirements in the local development environment immediately upon completion, before being considered ready for integration with previously developed components.
- **Adaptability to Change:** Several features of the system evolved substantially from their initial conception. The statistics page, for example, was originally planned as a simple numerical summary of task totals. During Sprint 5, the team decided to extend it significantly to include Chart.js-powered interactive bar and doughnut charts and a detailed priority breakdown table with animated progress bars, based on practical evaluation of what would provide the most user value.
- **Working Software as the Measure of Progress:** At the conclusion of each sprint, the team assessed progress exclusively in terms of working, demonstrable software functionality rather than lines of code written, documents produced, or hours invested. This focus kept the team's efforts consistently oriented toward delivering a usable product.
- **Simplicity and Minimalism:** The Agile principle of maximizing the work not done — favoring the simplest solution that satisfies the requirement over a more complex one that over-engineers it — was consistently applied. The decision to implement the frontend in vanilla JavaScript without a framework, for instance, was a direct application of this principle, eliminating build toolchain complexity and excessive dependency management overhead.
- **Collaborative Communication:** As a co-located three-member team, the project group was able to conduct frequent in-person design discussions, code reviews, and joint

debugging sessions, fulfilling the Agile emphasis on direct face-to-face communication as the most effective channel for conveying information within a development team.

3.2.2 Sprint Structure and Timeline

The complete development of Task Flow was organized into six sprints spanning the academic year 2025 to 2026. Each sprint had a clearly defined goal, a set of planned deliverables, and a duration of four to six weeks. The sprint structure is summarized in Table 3.1 below.

Table 3.1: Agile Sprint Summary for Task Flow Development

Sprint	Duration	Sprint Goal	Key Deliverables
Sprint 1	Weeks 1–5	Project Setup & Architecture	Requirements gathering, tech stack selection, PostgreSQL schema design, Node.js/Express.js server initialization, folder structure, environment configuration with dotenv.
Sprint 2	Weeks 6–10	Authentication System	User signup, login, logout, session management, bcrypt password hashing, forgot-password flow, auth middleware, and styled auth pages (index.html, signup.html, forgot-password.html).
Sprint 3	Weeks 11–16	Core Task Management	Full CRUD for tasks, task creation modal, task card UI with priority badges, due date display, overdue detection, pending page, completed page, task status toggling (complete/restore/delete).
Sprint 4	Weeks 17–21	Category Management & Sidebar	Custom category creation with color picker, sidebar category list rendering, category-based task filtering on the dashboard, category deletion, default category seeding for new users.
Sprint 5	Weeks 22–26	Statistics & Analytics	Statistics page with summary stat cards, Chart.js bar chart for priority distribution, doughnut chart for task status ratio, priority breakdown table with animated completion progress bars.
Sprint 6	Weeks 27–32	Polish, Testing & Documentation	Responsive mobile design, hamburger menu sidebar, toast notification system, rate limiting, XSS sanitization, cross-browser testing, bug fixes, and full thesis documentation.

3.3 Requirements Engineering

Requirements engineering is the systematic process of eliciting, specifying, validating, and managing the requirements of a software system. It serves as the bridge between stakeholder needs and the technical specifications that guide implementation. For Task Flow, requirements were gathered through a combination of self-elicitation — since the project team members were concurrently the primary target users of the system — competitive analysis of existing task management tools (as reviewed in Chapter 2), and a structured review of the project proposal submitted at the commencement of the Final Year Project.

Requirements are classified into two fundamental categories: functional requirements, which describe the specific behaviors and capabilities the system must provide, and non-functional requirements, which define the quality attributes and operational constraints the system must satisfy. Both sets of requirements are documented in full detail below.

3.3.1 Functional Requirements

Functional requirements define what the Task Flow system must do. Each requirement is assigned a unique identifier (FR-XX) and a priority level — High (critical to core functionality), Medium (important but not blocking), or Low (desirable enhancement).

Table 3.2: Functional Requirements of Task Flow

ID	Functional Requirement Description	Priority
FR-01	The system shall allow new users to register by providing full name, email address, and a password of minimum six characters.	High
FR-02	The system shall validate email uniqueness and format during registration and return an appropriate error if the email is already registered.	High
FR-03	The system shall authenticate users via email and password, maintaining an authenticated session using a server-side session cookie upon successful login.	High
FR-04	The system shall allow users to log out, destroying the server-side session and clearing the session cookie.	High
FR-05	The system shall provide a forgot-password feature enabling users to reset their password by verifying their registered email and submitting a new password.	High
FR-06	Unauthenticated requests to any protected page shall result in automatic redirection to the login page.	High
FR-07	Authenticated users shall be able to create tasks with a mandatory title,	High

ID	Functional Requirement Description	Priority
	optional description, optional due date, priority (high/medium/low), and optional category.	
FR-08	The dashboard shall display all pending tasks for the authenticated user, supporting search by title and filtering by priority and category.	High
FR-09	Users shall be able to edit the title, description, due date, priority, and category of any task through a modal dialog.	High
FR-10	Users shall be able to delete a task after confirmation through a dedicated delete confirmation modal to prevent accidental deletion.	High
FR-11	Users shall be able to mark a pending task as complete, with the completion timestamp automatically recorded in the database.	High
FR-12	Users shall be able to restore a completed task to pending status, clearing the stored completion timestamp.	High
FR-13	The system shall automatically detect overdue tasks and visually highlight them with a distinct warning style and icon on all task list pages.	High
FR-14	A dedicated Completed Tasks page shall list all tasks marked complete, showing the completion date on each task card.	Medium
FR-15	A dedicated Pending Tasks page shall list all pending tasks with search and priority filter functionality, in addition to the main dashboard view.	Medium
FR-16	Users shall be able to create custom task categories with a name and a color selected via a color picker input.	Medium
FR-17	User-defined categories shall appear in the sidebar navigation and enable task filtering on the dashboard when clicked.	Medium
FR-18	Deleting a category shall set all tasks previously assigned to that category to an unassigned (null) state, preserving the tasks themselves.	Medium
FR-19	The Statistics page shall display summary stat cards for total tasks, completed tasks, pending tasks, and overall completion rate percentage.	Medium
FR-20	The Statistics page shall render an interactive Chart.js bar chart showing the count of high, medium, and low priority tasks.	Medium
FR-21	The Statistics page shall render an interactive Chart.js doughnut chart displaying the ratio of completed to pending tasks.	Medium
FR-22	A priority breakdown table on the Statistics page shall display per-priority totals, completed counts, pending counts, and animated completion percentage progress bars.	Medium
FR-23	The dashboard shall display a time-sensitive greeting (Good morning/afternoon/evening) using the authenticated user's name and the current date.	Low
FR-24	The sidebar Pending navigation link shall display a real-time badge	Low

ID	Functional Requirement Description	Priority
	showing the current count of pending tasks for the authenticated user.	
FR-25	The system shall display auto-dismissing toast notification messages for all user actions, including task creation, editing, deletion, login, logout, and password reset outcomes.	Low

3.3.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, constraints, and operational characteristics that Task Flow must satisfy. These requirements define how the system performs its functions rather than what functions it performs.

Table 3.3: Non-Functional Requirements of Task Flow

ID	Category	Requirement Description
NFR-01	Security	User passwords shall be stored exclusively as bcrypt hashes with a salt factor of 10. Plain-text passwords shall never be stored, logged, or transmitted.
NFR-02	Security	Authentication endpoints shall be rate-limited to 200 requests per 15-minute window in development and 40 in production, protecting against brute-force attacks.
NFR-03	Security	All user-generated content rendered in the browser shall be sanitized using an escapeHtml() function before DOM insertion, preventing stored XSS attacks.
NFR-04	Security	Session cookies shall be set with httpOnly and sameSite: strict attributes to prevent client-side script access and cross-site request forgery.
NFR-05	Security	In production, session cookies shall additionally carry the secure flag, ensuring transmission exclusively over HTTPS connections.
NFR-06	Security	All database queries shall use parameterized inputs via the pg library's query function, preventing SQL injection attacks.
NFR-07	Performance	API responses shall be delivered within an acceptable latency for a locally hosted application. Indexed foreign key columns on tasks and categories tables shall optimize frequent queries.
NFR-08	Usability	The interface shall be intuitive enough for a new user to create their first task within three minutes of account registration, without requiring any prior training.
NFR-09	Usability	Immediate, clear feedback shall be provided to users through auto-dismissing toast notifications for all create, update, delete, and authentication operations.
NFR-	Responsiveness	The application shall function correctly across all screen widths from

ID	Category	Requirement Description
10		320px (small mobile) to 2560px (large desktop) using CSS media queries and flexible layout techniques.
NFR-11	Responsiveness	On mobile devices, the sidebar shall collapse and be accessible via a hamburger menu in the topbar, with a semi-transparent overlay disabling background interaction when open.
NFR-12	Maintainability	The codebase shall follow a modular folder structure separating frontend pages, backend routes, middleware, database configuration, and SQL schema files into dedicated directories.
NFR-13	Reliability	All API endpoints shall implement try-catch error handling, returning appropriate HTTP status codes (400, 401, 404, 409, 500) and descriptive JSON error messages for all failure cases.
NFR-14	Compatibility	The application shall function correctly in all modern browsers including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari, without browser plugins.
NFR-15	Data Integrity	The database schema shall enforce referential integrity via foreign key constraints, with cascading deletes ensuring that user removal also removes all associated tasks and categories.

3.4 Technology Stack Selection

The selection of an appropriate technology stack is a foundational engineering decision with far-reaching implications for the performance, security, maintainability, and scalability of the resulting system. The Task Flow technology stack was chosen after evaluating alternatives against five criteria: open-source availability and licensing, quality of community documentation, suitability for full-stack web application development, component compatibility, and alignment with the development team's existing technical competencies.

3.4.1 Frontend Technologies

The frontend of Task Flow is implemented in HTML5, CSS3, and vanilla JavaScript. The choice to avoid a JavaScript framework such as React, Vue, or Angular was deliberate. The scope of the application — a multi-page web application with dynamic DOM manipulation, asynchronous API calls, and modal management — is fully achievable using native browser APIs without the overhead of a framework's build toolchain, component model, or virtual DOM. Building in vanilla JavaScript also reinforced the team's mastery of foundational web development skills.

The CSS layer makes extensive use of custom properties (CSS variables) to implement a cohesive design system. The Ocean Breeze color palette — centred on a primary blue (#4A90E2) with semantic colors for priority levels (red #EF4444 for high, amber #F59E0B for medium, green #10B981 for low) — is defined as root-level CSS variables, making the design trivially easy to modify or re-theme. CSS Flexbox and Grid handle all page layouts, ensuring natural responsiveness without a CSS utility framework.

Chart.js 4.4.1 is included via CDN exclusively on the statistics page, providing responsive, canvas-based bar and doughnut charts. Font Awesome 6.5.0 provides the icon set. The Plus Jakarta Sans Google Font provides the primary typeface, offering a clean and modern aesthetic suitable for a productivity application.

3.4.2 Backend Technologies

The backend runs on Node.js (v18+) with Express.js (v5.2.1). Node.js was selected for its non-blocking, event-driven I/O model — well-suited to the frequent, concurrent database operations of a task management API — its use of JavaScript (reducing context-switching between frontend and backend work), and its extensive npm package ecosystem. Express.js was chosen for its minimalist routing API and rich middleware ecosystem.

The backend dependency set is deliberately kept lean: express-session for server-side session management, bcryptjs for password hashing, express-rate-limit for brute-force protection on authentication endpoints, cors for origin access control, and dotenv for environment-variable-based configuration. Each dependency was selected for a specific, well-defined purpose, avoiding dependency bloat.

3.4.3 Database Technology

PostgreSQL (v14+) serves as the relational database management system. Its ACID compliance, support for UUID primary keys, check constraints on enumerated fields (priority: high/medium/low; status: pending/complete), cascading foreign key deletes, and robust indexing capabilities make it a technically appropriate choice for the structured, relational data model of Task Flow. The node-postgres (pg) library provides the database client, utilizing a connection pool (Pool) and parameterized query execution to prevent SQL injection.

Table 3.4: Technology Stack Summary

Layer	Technology	Version	Purpose in Task Flow
Frontend	HTML5	HTML5	Page structure and semantic markup for all application pages
Frontend	CSS3	CSS3	Styling, responsive layout, CSS design system, animations
Frontend	JavaScript	ES2020+	DOM manipulation, async API calls, client-side application logic
Frontend	Chart.js	4.4.1	Interactive bar and doughnut chart rendering on Statistics page
Frontend	Font Awesome	6.5.0	Icon library used throughout the user interface
Frontend	Plus Jakarta Sans	Web Font	Primary application typeface loaded from Google Fonts
Backend	Node.js	18+	Server-side JavaScript runtime environment
Backend	Express.js	5.2.1	Web framework for REST API routing and middleware management
Backend	express-session	1.19.0	Server-side session management and authentication state tracking
Backend	bcryptjs	3.0.2	Password hashing and verification using the bcrypt algorithm
Backend	express-rate-limit	8.4.1	Rate limiting middleware protecting authentication endpoints
Backend	cors	2.8.6	Cross-Origin Resource Sharing header management
Backend	dotenv	16.4.5	Environment variable management from .env configuration file
Database	PostgreSQL	14+	Primary relational database for persistent storage of all data
Database	pg (node-postgres)	8.13.1	PostgreSQL client providing connection pooling and parameterized queries

3.5 Development Tools and Environment

The development of Task Flow was carried out on Windows-based development machines. The following tools supported the development workflow across all three team members.

3.5.1 Code Editor and Version Control

Visual Studio Code was used as the primary integrated development environment, chosen for its first-class JavaScript and Node.js support, built-in terminal, Git integration, and rich extension marketplace. Git version control was used throughout the project with the repository hosted on GitHub, enabling the three-member team to manage parallel development streams, maintain a traceable history of all code changes, and facilitate code review before merging contributions. The `.gitignore` file excluded the `server/.env` configuration file containing sensitive database credentials and the session secret, the `node_modules` directory, and other environment-specific artifacts.

3.5.2 Database Administration and API Testing

pgAdmin 4 served as the graphical database administration tool for PostgreSQL, providing a visual interface for creating the taskflow database, executing the SQL schema migration, and inspecting table contents during development. The database migration script (`server/scripts/migrate.js`) was also invoked directly from the command line using the `npm run db:migrate` script defined in `server/package.json`, enabling reproducible database setup from a clean state.

API endpoint testing during backend development was performed using a combination of the browser developer tools Network panel — which provided real-time visibility into request and response payloads during frontend-driven testing — and direct invocation of JavaScript functions from the browser console for scenarios not yet wired to the UI. This integrated approach eliminated the need for a separate API testing application and kept the development feedback loop tight.

3.5.3 Deployment Configuration

The application was fully configured for deployment on Railway, a cloud platform offering native Node.js and PostgreSQL hosting. The Express server serves static frontend files from the project root directory, meaning the same server handles both API requests and frontend asset delivery. The CORS configuration in `server.js` includes an explicit allowlist supporting the Railway deployment URL (<https://task-flow-fyp-production.up.railway.app>) as well as additional origins configurable through the `CORS_ORIGINS` environment variable. Due to deployment challenges encountered during testing, the system is currently run locally at

http://localhost:3000, where the full application — frontend and API — is accessible through the browser without any configuration changes.

3.6 Security Architecture

Security was treated as a first-class design concern throughout the Task Flow project rather than being deferred to a post-implementation hardening phase. The security architecture addresses four primary threat categories: credential compromise, session hijacking, cross-site scripting, and SQL injection.

Credential security is achieved through bcrypt password hashing. Passwords are hashed with a salt factor of 10 immediately upon receipt at the server and the hash alone is persisted. The bcrypt compare function, which operates in constant time regardless of whether the password matches or not, is used for authentication checks, preventing timing-based side-channel attacks. Rate limiting on the /auth/login, /auth/signup, and /auth/forgot-password/* endpoints restricts excessive authentication attempts from any single IP address.

Session security is enforced through careful cookie configuration: the httpOnly flag prevents client-side JavaScript from accessing the session cookie, the sameSite: strict setting blocks the cookie from being sent in cross-site requests, and the secure flag (active in production) restricts cookie transmission to HTTPS connections. The SESSION_SECRET environment variable, required in production, signs the session cookie to prevent forgery.

XSS protection is implemented at the rendering layer through the escapeHtml() function in app.js, which replaces the five HTML special characters (&, <, >, ", ') with their safe HTML entity equivalents before any user-supplied data is inserted into the DOM. This ensures that even if a user submits a task title containing malicious JavaScript, it will be rendered as harmless text rather than executed as code.

SQL injection prevention is achieved through consistent use of parameterized queries in every database operation throughout the backend. The pg library's query(text, params) API ensures that user-supplied values are always bound as data parameters, never concatenated into SQL strings, rendering injection attacks structurally impossible regardless of the content of user input.

3.7 Project Team Roles and Responsibilities

Task Flow was developed by three final-year BSCS students working collaboratively under the supervision of Dr. Shahzad. While the small team size naturally led to overlap in design discussions, testing, and debugging activities, primary ownership of specific system components was distributed as follows:

- Farhan Khan (AWKUM-22147646) led the backend development, including the Express.js server architecture, RESTful API design and implementation across all three route modules (auth, tasks, categories), PostgreSQL database schema design and migration scripting, session-based authentication, security middleware integration (rate limiting, CORS configuration), and the overall client-server integration strategy. Farhan also coordinated the project's architectural decisions and served as the primary point of contact for technical integration issues across team deliverables.
- Kamran Khan (AWKUM-22157075) led the frontend visual design and implementation, including the HTML structure and CSS styling of all application pages, the responsive design system using CSS custom properties and Flexbox/Grid layouts, the mobile sidebar and topbar implementation, typography and iconography integration, and the overall visual consistency of the user interface across all pages and device sizes.
- Muneeb Diyar Khan (AWKUM-22147942) led the client-side JavaScript development in app.js, including the apiFetch helper function, login and signup handlers, task CRUD functions, task status toggling logic, category management functions, sidebar dynamic rendering, the pending task badge update mechanism, the toast notification system, the task card HTML builder function (buildTaskCard), and the Chart.js visualization integration on the statistics page.

All three team members participated equally in requirements gathering, sprint planning and review meetings, functional testing, bug identification and resolution, and the writing and review of this thesis document. The collaborative structure ensured that each team member acquired a comprehensive working knowledge of all components of the system, not only those in their primary area of responsibility.

CHAPTER 4

System Design

4.1 Overview of System Design

System design is the process of defining the architecture, components, interfaces, and data flows of a software system in order to satisfy the specified requirements. It serves as the technical blueprint that guides the implementation phase, ensuring that the resulting system is coherent, maintainable, and capable of fulfilling both functional and non-functional requirements. For Task Flow, the system design encompasses five primary dimensions: overall system architecture, database schema design, RESTful API design, user interface design, and security design.

The design of Task Flow follows a classical client-server architectural pattern, where a browser-based frontend communicates with a Node.js/Express.js backend over HTTP using a RESTful API. All application data is persisted in a PostgreSQL relational database managed exclusively by the backend. This separation of concerns — frontend presentation logic, backend business logic, and database persistence — promotes modularity and makes each layer independently testable and maintainable.

4.2 System Architecture

The Task Flow system is built on a three-tier architecture comprising a presentation tier (frontend), an application tier (backend), and a data tier (database). Each tier has clearly defined responsibilities and communicates with adjacent tiers through well-defined interfaces.

4.2.1 Presentation Tier (Frontend)

The presentation tier consists of six HTML pages served as static files directly by the Express.js server: `index.html` (login), `signup.html` (registration), `forgot-password.html` (password recovery), `dashboard.html` (main task view), `pending.html` (pending tasks), `complete.html` (completed tasks), and `stats.html` (statistics). All pages share a single CSS stylesheet (`css/style.css`) and a single JavaScript module (`js/app.js`), which contains all client-side logic including API communication, DOM manipulation, task rendering, modal management, category handling, and the toast notification system.

The frontend communicates with the backend exclusively through asynchronous HTTP requests using the Fetch API, targeting the RESTful API endpoints exposed at the `/api/*` path. All requests are made with credentials: include to ensure that session cookies are transmitted, enabling the server to identify the authenticated user across requests. The `apiFetch()` helper function in `app.js` abstracts the Fetch API call, handling JSON serialization of request bodies, parsing of JSON responses, and unified error propagation.

4.2.2 Application Tier (Backend)

The application tier is a Node.js application running the Express.js framework. The entry point is `server/server.js`, which initializes the Express application, configures middleware (CORS, JSON body parsing, URL-encoded body parsing, session management), registers the three route modules, configures the static file serving for the frontend, and starts the HTTP server on the configured port (default: 3000).

The backend is organized into three Express Router modules, each responsible for a distinct domain of the application. The authentication router (`server/routes/auth.js`) handles user registration, login, logout, session verification, and the two-step forgot-password flow. The tasks router (`server/routes/tasks.js`) handles all CRUD operations on tasks, status updates, and task retrieval. The categories router (`server/routes/categories.js`) handles category creation, retrieval, deletion, and default category seeding for new users. All three routers are protected by the `requireAuth` middleware (`server/middleware/auth.js`), which verifies the presence of a valid session before allowing access to any protected endpoint.

4.2.3 Data Tier (Database)

The data tier consists of a PostgreSQL database containing three tables: users, categories, and tasks. The database is accessed from the backend exclusively through the `node-postgres (pg)` library using a connection pool, ensuring efficient reuse of database connections. All database queries use parameterized inputs to prevent SQL injection. The database schema enforces data integrity through primary key constraints (UUID), foreign key constraints with cascading deletes, unique constraints on email addresses, and check constraints on enumerated fields (priority and status).

4.2.4 Architecture Diagram Description

The system architecture can be described as follows. A user interacts with the application through a web browser, which renders the HTML/CSS/JavaScript frontend served by the Express.js server. When a user performs an action — such as creating a task or logging in — the frontend JavaScript code issues an HTTP request to the appropriate `/api/*` endpoint on the same server. The Express.js server receives the request, validates the session through the auth middleware, routes the request to the appropriate handler in the relevant router module, executes the required database operation through the pg connection pool against the PostgreSQL database, and returns a JSON response to the frontend. The frontend then updates the DOM accordingly and displays a toast notification to the user.

4.3 Database Design

The database design for Task Flow follows the principles of relational database normalization to ensure data integrity, eliminate redundancy, and support efficient query execution. The schema consists of three tables organized around the core domain entities of the application: users, categories, and tasks.

4.3.1 Database Schema

The users table is the root entity of the data model. It stores the account information for each registered user and serves as the parent table for both the categories and tasks tables through foreign key relationships. The table contains the following columns: `id` (UUID, primary key, default `gen_random_uuid()`), `name` (TEXT, not null), `email` (TEXT, not null, unique), `password_hash` (TEXT, not null), and `created_at` (TIMESTAMPTZ, not null, default `now()`).

The categories table stores user-defined task categories with color assignments. Each category belongs to a single user, enforced by a foreign key constraint on the `user_id` column. The columns are: `id` (UUID, primary key), `user_id` (UUID, foreign key referencing `users.id` with `ON DELETE CASCADE`), `name` (TEXT, not null), `color` (TEXT, not null, default `'#4A90E2'`), and `created_at` (TIMESTAMPTZ, not null, default `now()`). The `ON DELETE CASCADE` constraint ensures that when a user account is deleted, all their categories are automatically removed.

The tasks table is the central entity of the application. Each task belongs to a user (via `user_id`) and optionally to a category (via `category_id`). The columns are: `id` (UUID, primary

key), user_id (UUID, foreign key referencing users.id with ON DELETE CASCADE), category_id (UUID, nullable foreign key referencing categories.id with ON DELETE SET NULL), title (TEXT, not null), description (TEXT, not null, default empty string), due_date (DATE, nullable), priority (TEXT, not null, default 'medium', check constraint: IN ('high', 'medium', 'low')), status (TEXT, not null, default 'pending', check constraint: IN ('pending', 'complete')), created_at (TIMESTAMPTZ, not null, default now()), and completed_at (TIMESTAMPTZ, nullable). The ON DELETE SET NULL on category_id ensures that deleting a category does not delete the associated tasks but simply removes their category assignment.

Table 4.1: Database Tables and Column Descriptions

Table	Column	Data Type	Description and Constraints
users	id	UUID	Primary key, auto-generated using gen_random_uuid()
users	name	TEXT	Full name of the registered user. Cannot be null.
users	email	TEXT	Email address. Unique constraint prevents duplicate registrations.
users	password_hash	TEXT	bcrypt hash of the user password. Plain-text password never stored.
users	created_at	TIMESTAMPTZ	Account creation timestamp. Defaults to current UTC time.
categories	id	UUID	Primary key, auto-generated.
categories	user_id	UUID	Foreign key to users.id. ON DELETE CASCADE removes user categories on account deletion.
categories	name	TEXT	Category name (max 50 chars enforced at application layer).
categories	color	TEXT	Hex color string (e.g., #4A90E2). Defaults to primary blue.
tasks	id	UUID	Primary key, auto-generated.
tasks	user_id	UUID	Foreign key to users.id. ON DELETE CASCADE removes user tasks on account deletion.
tasks	category_id	UUID	Nullable foreign key to categories.id. ON DELETE SET NULL preserves tasks when category deleted.

Table	Column	Data Type	Description and Constraints
tasks	title	TEXT	Task title. Mandatory field, validated at application layer.
tasks	description	TEXT	Optional task description. Defaults to empty string.
tasks	due_date	DATE	Optional due date. Stored as date-only value. Used for overdue detection.
tasks	priority	TEXT	Enumerated: 'high', 'medium', or 'low'. Check constraint enforced at DB level.
tasks	status	TEXT	Enumerated: 'pending' or 'complete'. Check constraint enforced at DB level.
tasks	completed_at	TIMESTAMPZ	Null for pending tasks. Set to now() when status changes to complete; cleared on restore.

4.3.2 Database Indexes

To optimize the performance of the most frequent query patterns, the following indexes are defined in the schema: `idx_categories_user_id` on `categories(user_id)`, `idx_tasks_user_id` on `tasks(user_id)`, and `idx_tasks_category_id` on `tasks(category_id)`. These indexes accelerate the retrieval of all categories and tasks belonging to a specific user — the most common operation in the application — as well as the lookup of tasks by category for filtering operations.

4.4 RESTful API Design

The Task Flow backend exposes a RESTful API organized under the `/api` base path. The API follows REST conventions: resources are identified by URI paths, HTTP methods convey the intended operation (GET for retrieval, POST for creation, PUT for full update, PATCH for partial update, DELETE for removal), and all request and response bodies use JSON encoding. Every API endpoint (except the authentication endpoints) requires an authenticated session, enforced by the `requireAuth` middleware.

Table 4.2: API Endpoints Summary

Method	Endpoint	Description
POST	<code>/api/auth/signup</code>	Register a new user account. Validates name, email uniqueness, and password length. Returns user object and creates session.
POST	<code>/api/auth/login</code>	Authenticate user with email and password. Returns user object and creates session on success.

Method	Endpoint	Description
POST	/api/auth/logout	Destroy the current user session and clear the session cookie.
GET	/api/auth/me	Return the currently authenticated user object from the session. Used by the frontend to verify auth state on page load.
POST	/api/auth/forgot-password/check-email	Step 1 of password reset: check whether the provided email is registered. Returns {exists: true/false}.
POST	/api/auth/forgot-password/set-password	Step 2 of password reset: update the password for the given email after re-verification. Returns success message.
GET	/api/tasks	Retrieve all tasks belonging to the authenticated user, ordered by creation date descending. Auth required.
POST	/api/tasks	Create a new task for the authenticated user. Requires title; optional fields: description, dueDate, priority, category. Auth required.
PUT	/api/tasks/:id	Update all editable fields (title, description, dueDate, priority, categoryId) of a specific task. Auth and ownership required.
PATCH	/api/tasks/:id/status	Update only the status field of a task (pending or complete). Sets or clears completed_at timestamp accordingly. Auth required.
DELETE	/api/tasks/:id	Permanently delete a specific task. Auth and ownership required. Returns 404 if task not found or not owned by user.
GET	/api/categories	Retrieve all categories for the authenticated user. Seeds default categories (Work, Office, Personal, Health) if none exist.
POST	/api/categories	Create a new category with the provided name and color. Name is validated (non-empty, max 50 chars). Auth required.
DELETE	/api/categories/:id	Delete a category. Tasks assigned to the deleted category have category_id set to null (ON DELETE SET NULL). Auth required.
GET	/api/ping	Health check endpoint. Returns {status: "ok"} with a server running confirmation message. No auth required.

All API responses follow a consistent JSON structure. Successful responses include a message field and, where applicable, a data field containing the created or updated resource (e.g., {message: "Task created!", task: {...}}). Error responses include an error field with a descriptive message (e.g., {error: "Email already registered."}). HTTP status codes are used semantically: 200 for successful retrieval and updates, 201 for successful creation, 400 for

invalid input, 401 for unauthenticated requests, 404 for resource not found, 409 for conflicts (duplicate email), and 500 for server errors.

4.5 User Interface Design

The user interface of Task Flow was designed with a focus on clarity, visual hierarchy, and ease of use. The design system is built around a cohesive color palette, consistent typography, and a set of reusable UI components defined through CSS custom properties. This section presents the key interface screens of the application with annotations describing the design decisions made for each.

4.5.1 Login Page

The login page adopts a split-panel layout. The left panel features a gradient blue background (transitioning from the deep primary color #1D5FA8 to a lighter #6BAED6) with the Task Flow brand logo, application name, and a motivational tagline. Decorative circular elements create visual depth without distracting from the primary call-to-action. The right panel presents a clean, minimal login form with labeled email and password input fields, a forgot-password link, and a gradient login button. This split layout is a well-established pattern in authentication UI design that effectively separates the marketing message from the functional form.

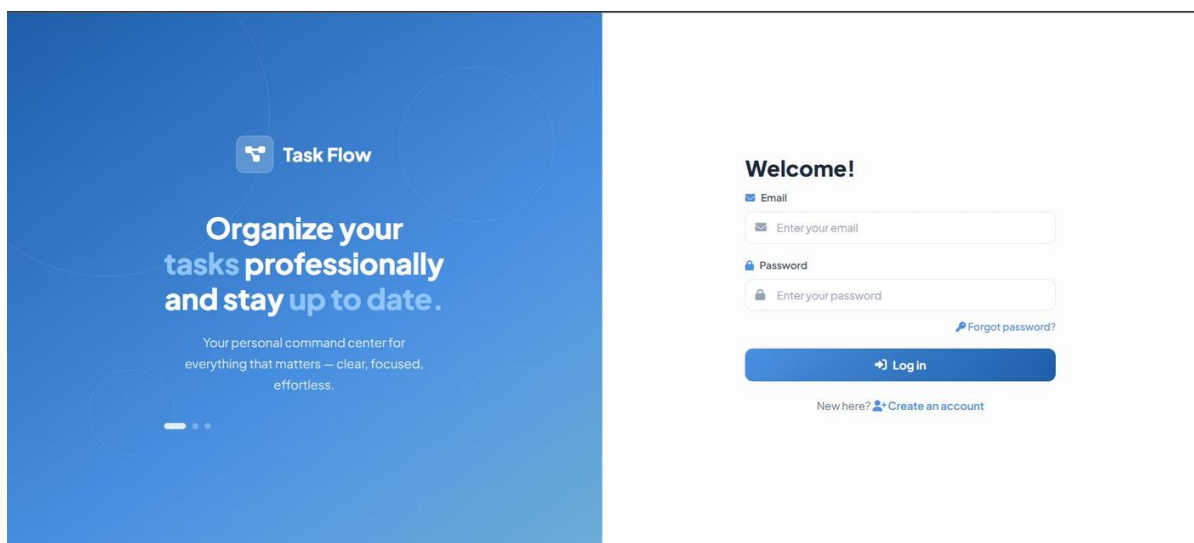


Figure 4.1: Task Flow Login Page

4.5.2 Registration Page

The signup page follows the same split-panel layout as the login page, maintaining visual consistency across the authentication flow. The left panel on the signup page features feature highlights — Smart Task Management, Progress Tracking & Stats, and Custom Categories — presented as frosted-glass cards, giving prospective users an immediate understanding of the system's value proposition. The right panel contains a four-field registration form (full name, email, password, confirm password) with a prominent Sign Up button. Input validation is performed client-side before submission, with toast notifications providing immediate error feedback for issues such as mismatched passwords or insufficient password length.

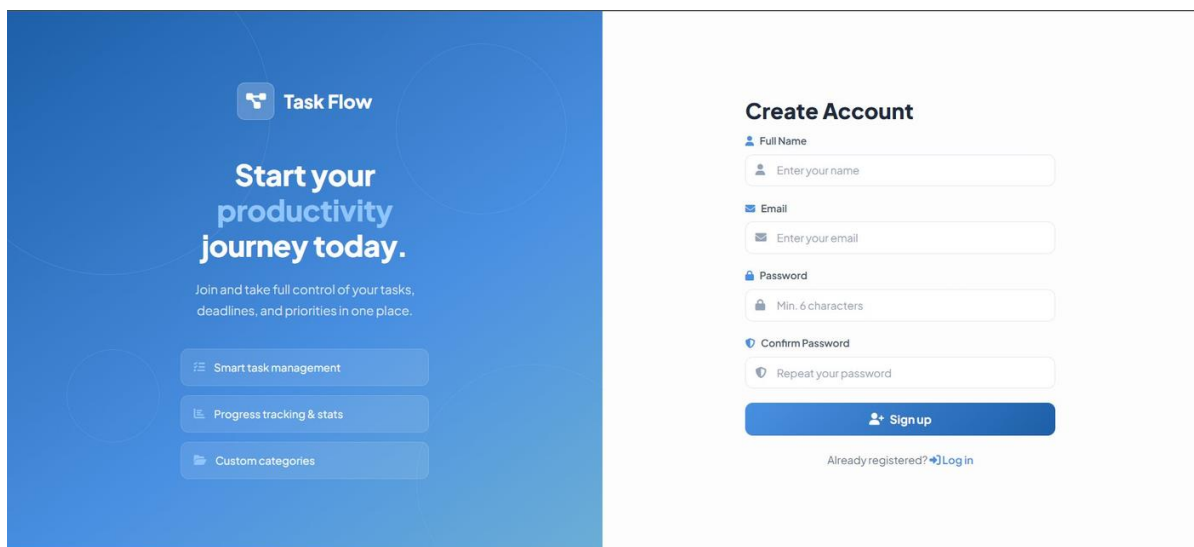
The image shows a split-panel registration page for 'Task Flow'. The left panel has a blue background with the 'Task Flow' logo at the top. Below the logo, it says 'Start your productivity journey today.' and 'Join and take full control of your tasks, deadlines, and priorities in one place.' There are three frosted-glass cards: 'Smart task management', 'Progress tracking & stats', and 'Custom categories'. The right panel is white and titled 'Create Account'. It contains four input fields: 'Full Name' (with a person icon), 'Email' (with an envelope icon), 'Password' (with a lock icon and a 'Min. 6 characters' hint), and 'Confirm Password' (with a checkmark icon and a 'Repeat your password' hint). A blue 'Signup' button with a person icon is at the bottom. Below the button, it says 'Already registered? * Login'.

Figure 4.2: Task Flow Sign Up / Registration Page

4.5.3 Dashboard

The dashboard is the primary interface of the application and is designed around a persistent sidebar navigation on the left and a main content area on the right. The sidebar contains the application logo, navigation links (Dashboard, Stats, Tasks with expandable sub-links for Pending and Completed), a categorized task filter section, and a user profile card at the bottom displaying the user's name, email, and a logout button. The main content area features a personalized time-sensitive greeting displaying the user's name and the current date, followed by a search bar and an Add New Task button in the action bar. A filter bar beneath allows users to switch between Pending and All tasks and to filter by priority level using a dropdown. Tasks are rendered as cards in a vertical list below the filter bar.

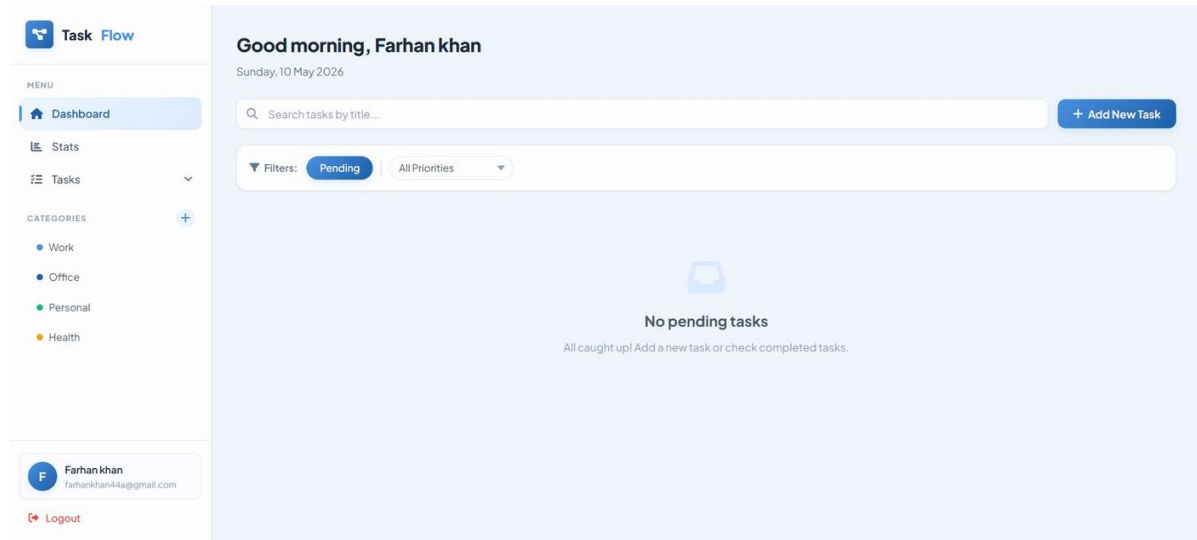
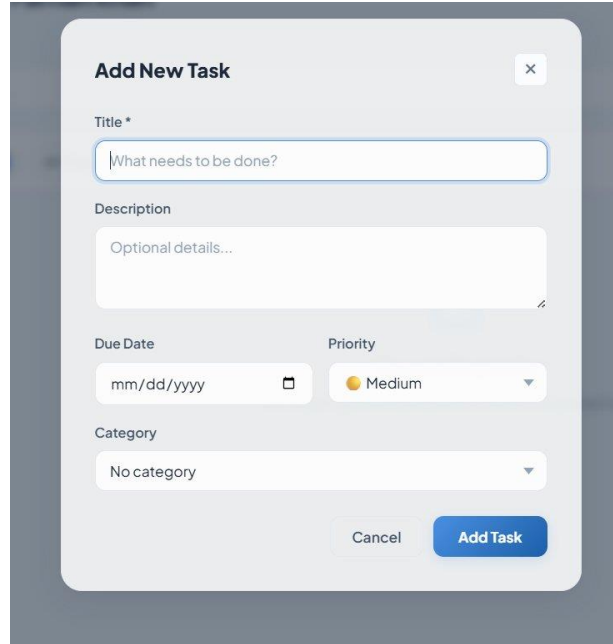


Figure 4.3: Task Flow Dashboard — Main Task View

4.5.4 Add New Task Modal

Task creation is handled through a modal dialog that overlays the current page with a blurred backdrop. The modal contains five input fields: a mandatory title field with placeholder text "What needs to be done?", an optional description textarea, a due date picker, a priority dropdown (Low/Medium/High, defaulting to Medium), and a category dropdown populated from the user's defined categories. The modal is triggered by the Add New Task button and can be dismissed by clicking the close icon, the Cancel button, pressing the Escape key, or clicking the backdrop overlay. The same modal is reused for task editing, with the title changing to "Edit Task" and all fields pre-populated with the existing task data.

A modal dialog titled "Add New Task" with a close button (X) in the top right corner. The form contains a "Title *" field with placeholder text "What needs to be done?", a "Description" field with placeholder text "Optional details...", a "Due Date" field with a date input "mm/dd/yyyy" and a calendar icon, a "Priority" dropdown menu currently showing "Medium" with a yellow dot icon, and a "Category" dropdown menu currently showing "No category". At the bottom right are "Cancel" and "Add Task" buttons.

Add New Task [X]

Title *

What needs to be done?

Description

Optional details...

Due Date

mm/dd/yyyy [Calendar Icon]

Priority

Medium [Yellow Dot Icon]

Category

No category [Dropdown Arrow]

Cancel Add Task

Figure 4.4: Add New Task Modal Dialog

4.5.5 Statistics Page

The statistics page provides a comprehensive view of the user's productivity metrics. At the top, four summary stat cards display total tasks, completed tasks, pending tasks, and the overall completion rate percentage. Each card features a color-coded icon, the numeric value in large bold font, and a contextual sub-label. Below the stat cards, two charts are displayed side-by-side: a bar chart showing the count of high, medium, and low priority tasks (with priority-appropriate colors: red, amber, green), and a doughnut chart showing the ratio of completed to pending tasks. Each chart has a legend below it for accessibility. At the bottom of the page, the Priority Breakdown table provides a detailed tabular view with animated progress bars showing the completion percentage for each priority level.

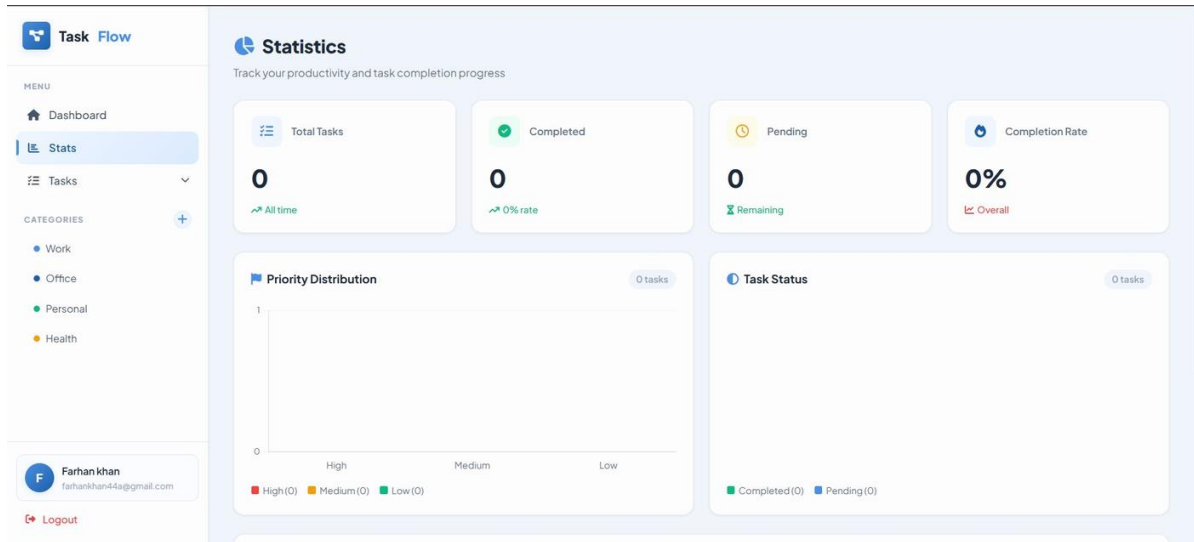


Figure 4.5: Statistics Page — Summary Cards and Charts

Priority Breakdown					View all tasks →
PRIORITY	TOTAL	COMPLETED	PENDING	COMPLETION %	
High	0	0	0		0%
Medium	0	0	0		0%
Low	0	0	0		0%

Figure 4.6: Statistics Page — Priority Breakdown Table

4.5.6 Task Card Design

Each task is rendered as a card component with consistent structure across all task list pages. The card displays the task title (struck through for completed tasks), an optional description, a priority badge (color-coded with an appropriate icon), an optional category tag (styled in the category's chosen color), and a due date indicator (shown in red with a warning icon for overdue tasks). The action buttons — Edit, Delete, and Complete (or Restore for completed tasks) — are aligned to the bottom right of the card. Cards have a subtle border, rounded corners, a light shadow, and a hover state that lifts the card slightly and deepens the shadow, providing clear interactive feedback.

4.5.7 Responsive Design

The application is fully responsive across device sizes. On screens narrower than 900px, the sidebar is hidden by default and slides in from the left when the hamburger menu button in the topbar is tapped. A semi-transparent overlay with a blur effect covers the main

content area when the sidebar is open, preventing accidental interaction with the background. The topbar displays the Task Flow logo and the hamburger button. Layout grids for the statistics page reflow from two-column to one-column at 900px, and several UI elements — including action bars and modal action buttons — stack vertically on small screens.

4.6 Application Page Flow and Navigation

The navigation flow of the Task Flow application is designed to be intuitive and consistent. The login page (`index.html`) is the entry point for unauthenticated users. Upon successful login or registration, users are redirected to the dashboard. All protected pages — dashboard, stats, pending, complete — check for an authenticated session on load and redirect unauthenticated visitors back to the login page.

The sidebar navigation, present on all authenticated pages, provides direct access to the Dashboard, Stats, and the Tasks submenu (expanding to reveal Pending and Completed sub-links). Category chips in the sidebar navigate to the dashboard filtered by the selected category. The topbar, visible only on mobile and tablet screens, provides the hamburger menu toggle. All pages include a Logout button in the sidebar footer that terminates the session and redirects to the login page.

The forgot-password flow is a two-step process on a dedicated page (`forgot-password.html`). In step one, the user enters their registered email. If the email exists in the database, the password entry form is revealed. In step two, the user sets and confirms a new password. On success, they are redirected back to the login page with a success toast notification. This flow does not require any email sending capability, making it self-contained and operable without external email service configuration.

4.7 Security Design

The security design of Task Flow addresses the primary threat categories identified during the requirements engineering phase. The authentication system uses bcrypt password hashing (salt rounds: 10), server-side session management with `httpOnly` and `sameSite: strict` cookie attributes, and rate limiting on all authentication endpoints. The data access layer uses parameterized PostgreSQL queries exclusively, eliminating SQL injection vulnerabilities. The

frontend rendering layer sanitizes all user-generated content through the `escapeHtml()` function before DOM insertion, preventing stored XSS attacks.

User data isolation is enforced at the database query level — every task and category query includes a `WHERE user_id = $userId` condition, ensuring that a user can only ever access or modify their own data, regardless of the task or category ID provided in the request URL. This prevents horizontal privilege escalation attacks where an authenticated user could access another user's data by guessing or enumerating resource IDs.

The Express.js server is configured to trust the proxy headers set by Railway (the deployment platform) in production, enabling correct IP address extraction for rate limiting when the application is running behind a reverse proxy. In development, private LAN IP addresses are accepted by the CORS configuration, facilitating testing from multiple devices on the local network without requiring changes to the application code.

CHAPTER 5

Implementation

5.1 Overview of the Implementation Phase

The implementation phase translates the system design specifications described in Chapter 4 into working, executable software. This chapter provides a detailed account of how each major component of the Task Flow system was built, including the project folder structure, the backend server and API implementation, the database setup and schema execution, the authentication module, the task and category management modules, the frontend page implementations, and the statistics and analytics engine. Where appropriate, representative code excerpts are included to illustrate key implementation decisions and patterns. The complete codebase is organized into a clear, modular folder structure that separates concerns between the frontend, backend, and database layers.

The implementation followed the Agile sprint plan described in Chapter 3, proceeding iteratively from the foundational server and authentication infrastructure through to the task management core, category system, analytics module, and finally the UI polish and responsive design. Each sprint produced a functional, testable software increment that was verified against the requirements before development of the next sprint commenced.

5.2 Project Structure and Organization

The Task Flow project is organized into a root directory containing all frontend HTML pages and assets, and a server subdirectory containing all backend Node.js code. This flat frontend structure — where HTML files reside at the root — allows Express.js to serve them as static files from the same origin as the API, eliminating CORS complexity for same-origin requests and allowing a single deployment artifact to serve both the frontend and the API.

```
task-flow/
  index.html           # Login page
  signup.html          # Registration page
  forgot-password.html # Password recovery page
  dashboard.html       # Main task dashboard
  pending.html         # Pending tasks page
  complete.html        # Completed tasks page
  stats.html           # Statistics and analytics page
  reset-password.html  # Redirect shim to forgot-password
  css/
```

```
style.css           # Unified stylesheet for all pages
js/
  app.js            # Shared client-side JavaScript module
server/
  server.js         # Express application entry point
  db.js             # PostgreSQL connection pool
  .env              # Environment variables (not in Git)
  .env.example      # Environment variable template
  package.json      # Backend dependencies and scripts
  routes/
    auth.js         # Authentication API routes
    tasks.js        # Task management API routes
    categories.js   # Category management API routes
  middleware/
    auth.js         # requireAuth middleware
  scripts/
    migrate.js      # Database schema migration script
  sql/
    schema.sql      # PostgreSQL DDL schema definition
```

Figure 5.1: Task Flow Project Folder Structure

This structure enforces a clear separation of concerns. All frontend code (HTML, CSS, JavaScript) resides outside the server directory and is served as static content. All backend code is encapsulated within the server directory. The routes directory separates API concerns into three domain-focused modules. The sql directory contains the canonical database schema definition that can be applied to any PostgreSQL instance to reproduce the database structure.

5.3 Backend Server Implementation

The backend server is implemented in `server/server.js`, which serves as the application entry point. The file performs several critical initialization tasks in sequence: loading environment variables from the `.env` file using `dotenv`, creating the Express application instance, configuring the trust proxy setting for production deployments behind a reverse proxy, setting up CORS with an explicit origin allowlist, registering JSON and URL-encoded body parsing middleware, configuring the `express-session` middleware with security-appropriate cookie settings, registering the three API route modules, configuring static file serving for the frontend, defining the health check endpoint, verifying the PostgreSQL database connection, and starting the HTTP server on the configured port.

5.3.1 Server Initialization and Middleware Stack

The middleware stack is configured in a specific order that reflects the request processing pipeline. CORS headers must be set before any route handlers execute, so the cors middleware is registered first. Body parsing middleware follows, enabling Express to parse JSON request bodies and URL-encoded form data. The session middleware is registered next, initializing the session system and making the req.session object available to all subsequent handlers. Finally, the static file serving middleware is registered, allowing the Express server to serve frontend HTML, CSS, and JavaScript files directly from the project root directory.

```
// Session configuration (server/server.js)
app.use(session({
  secret: process.env.SESSION_SECRET || "taskflow-secret-key-fyp-dev",
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: isProduction,          // HTTPS only in production
    httpOnly: true,                // No client-side JS access
    sameSite: "strict",           // CSRF protection
    maxAge: 1000 * 60 * 60 * 24 // 24-hour session lifetime
  }
}));
```

Figure 5.2: Session Middleware Configuration

5.3.2 Database Connection Module

The database connection is managed through a dedicated module (server/db.js) that creates and exports a PostgreSQL connection pool using the pg library. The module implements intelligent SSL configuration logic: connections to localhost or 127.0.0.1 default to no SSL (appropriate for local development), while connections to remote hosts default to SSL with configurable certificate verification (appropriate for hosted PostgreSQL instances such as those provided by Railway). This dual-mode SSL behavior eliminates the need to manually change connection settings between development and production environments.

```
// Database connection pool (server/db.js)
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,           // Maximum pool connections
  idleTimeoutMillis: 30000, // Close idle connections after 30s
  ...(ssl !== false ? { ssl } : {})
});

function query(text, params) {
  return pool.query(text, params); // Parameterized queries only
}
```

```
}  
  
module.exports = { pool, query };
```

Figure 5.3: PostgreSQL Connection Pool Implementation

5.4 Database Implementation

The PostgreSQL database schema is defined in `server/sql/schema.sql` and applied to the database using the migration script `server/scripts/migrate.js`, which can be executed via `npm run db:migrate` from the server directory. The schema uses `IF NOT EXISTS` clauses on all `CREATE TABLE` and `CREATE INDEX` statements, making the migration script idempotent — it can be run multiple times against the same database without producing errors or duplicating schema objects.

```
-- PostgreSQL Schema (server/sql/schema.sql)  
CREATE TABLE IF NOT EXISTS users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        TEXT NOT NULL,  
  email       TEXT NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE IF NOT EXISTS categories (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  name        TEXT NOT NULL,  
  color       TEXT NOT NULL DEFAULT '#4A90E2',  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE IF NOT EXISTS tasks (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  category_id UUID REFERENCES categories(id) ON DELETE SET NULL,  
  title       TEXT NOT NULL,  
  description  TEXT NOT NULL DEFAULT '',  
  due_date    DATE,  
  priority     TEXT NOT NULL DEFAULT 'medium'  
              CHECK (priority IN ('high', 'medium', 'low')),  
  status      TEXT NOT NULL DEFAULT 'pending'  
              CHECK (status IN ('pending', 'complete')),  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  completed_at TIMESTAMPTZ  
);
```

```
CREATE INDEX IF NOT EXISTS idx_categories_user_id ON
categories(user_id);
CREATE INDEX IF NOT EXISTS idx_tasks_user_id      ON tasks(user_id);
CREATE INDEX IF NOT EXISTS idx_tasks_category_id  ON
tasks(category_id);
```

Figure 5.4: PostgreSQL Schema Definition

The schema design makes several deliberate implementation choices. UUID primary keys are generated server-side by PostgreSQL using the `gen_random_uuid()` function, producing cryptographically random identifiers that are non-sequential and therefore not susceptible to enumeration attacks. The CHECK constraints on the priority and status columns enforce enumerated value restrictions at the database level, providing a second layer of validation below the application-level checks in the route handlers. The TIMESTAMPTZ (timestamp with timezone) type is used for all timestamp columns, ensuring that dates and times are stored with UTC timezone information regardless of the server's local timezone setting.

5.5 Authentication Module Implementation

The authentication module (`server/routes/auth.js`) implements six routes covering the complete user identity lifecycle: registration, login, logout, session verification, and the two-step password recovery flow. All routes in this module apply rate limiting middleware to protect against automated attack attempts.

5.5.1 User Registration

The signup route validates that all required fields are present, that the password meets the minimum length requirement of six characters, and that the email address conforms to a valid format using a regular expression. The password is then hashed using `bcryptjs` with a salt round factor of 10 before being stored. If the INSERT operation fails with a PostgreSQL error code 23505 (unique constraint violation), the route returns a 409 Conflict response with an appropriate error message informing the user that the email address is already registered. On successful registration, the new user object is stored in the session and a 201 Created response is returned.

```
// User registration (server/routes/auth.js)
router.post("/signup", signupLoginLimiter, async (req, res) => {
```

```

const { name, email, password } = req.body;
// Validate inputs
if (!name || !email || !password)
  return res.status(400).json({ error: "All fields are required."
});
if (password.length < 6)
  return res.status(400).json({ error: "Password min 6 characters."
});
const normalizedEmail = email.toLowerCase().trim();
// Hash password and insert user
const passwordHash = await bcrypt.hash(password, 10);
const result = await query(
  `INSERT INTO users (name, email, password_hash)
  VALUES ($1, $2, $3) RETURNING id, name, email`,
  [name.trim(), normalizedEmail, passwordHash]
);
req.session.user = safeUser(result.rows[0]);
res.status(201).json({ user: req.session.user });
});

```

Figure 5.5: User Registration Route Implementation

5.5.2 User Login and Session Management

The login route retrieves the user record by email (normalized to lowercase to ensure case-insensitive matching), then uses `bcrypt.compare()` to verify the submitted password against the stored hash. The `bcrypt.compare()` function performs a constant-time comparison, meaning it takes the same amount of time whether the password matches or not, preventing timing-based attacks that could reveal whether an email address is registered. On successful authentication, the user object (containing only `id`, `name`, and `email` — never the password hash) is stored in the session, and a success response is returned. Failed login attempts always receive the same generic error message ("Invalid email or password") regardless of whether the failure was due to an unrecognized email or an incorrect password, preventing user enumeration.

5.5.3 Authentication Middleware

The `requireAuth` middleware (`server/middleware/auth.js`) is applied to all task and category routes. It checks for the presence of `req.session` and `req.session.user` before allowing the request to proceed. If either is absent, it returns a 401 Unauthorized response. This middleware is registered at the router level rather than on individual routes, ensuring that it applies to every endpoint in the protected route modules without the need for repetitive per-route middleware declarations.

```
// Authentication middleware (server/middleware/auth.js)
function requireAuth(req, res, next) {
  if (!req.session || !req.session.user) {
    return res.status(401).json({ error: "Unauthorized. Please log
in." });
  }
  next();
}
module.exports = requireAuth;
```

Figure 5.6: Authentication Middleware Implementation

5.5.4 Password Recovery Flow

The forgot-password flow is implemented as two sequential API endpoints. The first endpoint (POST /api/auth/forgot-password/check-email) accepts an email address and queries the users table to determine whether an account with that email exists. It returns a JSON object with a boolean exists field. Crucially, this endpoint returns a 200 OK response regardless of whether the email exists or not, preventing email enumeration through response code analysis. The frontend uses the exists value to conditionally reveal the password entry form.

The second endpoint (POST /api/auth/forgot-password/set-password) accepts the email and new password, re-verifies that the email exists in the database (a second check is necessary because the two steps are independent HTTP requests), hashes the new password with bcrypt, and updates the password_hash column for the matching user. This approach allows password recovery without requiring any email sending infrastructure, which simplifies deployment significantly.

5.6 Task Management Module Implementation

The task management module (server/routes/tasks.js) implements the five task API endpoints: list all tasks, create a task, update a task, update task status, and delete a task. All routes are protected by the requireAuth middleware, and all database queries include a user_id = req.session.user.id condition to enforce data isolation between users.

5.6.1 Task Creation

The task creation route accepts the task data from the request body and applies several normalization and validation steps before the database INSERT. The priority field defaults to "medium" if not provided or if an invalid value is submitted. The due date is trimmed to the first 10 characters (YYYY-MM-DD format), preventing any timestamp components from

being stored in the DATE column. The category ID is set to null if the submitted value is an empty string, representing an unassigned task. The INSERT query uses the PostgreSQL RETURNING clause to return the complete newly created task row, which is then mapped through the rowToTask() helper function and returned to the client.

```
// Task creation (server/routes/tasks.js)
router.post("/", async (req, res) => {
  const { title, description, dueDate, priority, category } =
    req.body;
  const userId = req.session.user.id;
  if (!title || title.trim() === "")
    return res.status(400).json({ error: "Task title is required." });
  const pri = isValidPriority(priority) ? priority : "medium";
  const due = dueDate ? String(dueDate).slice(0, 10) : null;
  const catId = category && category.trim() !== "" ? category : null;
  const result = await query(
    `INSERT INTO tasks
      (user_id, category_id, title, description, due_date, priority,
status)
      VALUES ($1,$2,$3,$4,$5,$6,"pending")
      RETURNING *`,
    [userId, catId, title.trim(), description?.trim() || "", due, pri]
  );
  res.status(201).json({ task: rowToTask(result.rows[0]) });
});
```

Figure 5.7: Task Creation Route Implementation

5.6.2 Task Status Update and Overdue Detection

The task status update endpoint (PATCH /api/tasks/:id/status) accepts a status value of either "pending" or "complete" and updates the task record accordingly. A key feature of this endpoint is its handling of the completed_at timestamp: when a task is marked complete, the timestamp is set to the current time using PostgreSQL's now() function through a CASE expression in the UPDATE query. When a task is restored to pending, the completed_at field is set back to NULL. This server-side timestamp management ensures that completion times are accurate regardless of client-side clock settings.

Overdue detection is implemented entirely on the client side in the isOverdue() function in app.js. This function accepts a task's due date and status and returns true if the due date has passed (considering the end of the due date day, i.e., 23:59:59) and the task is still in pending status. This client-side approach avoids the need for a server-side scheduled job to update task

states and means that overdue status is always computed relative to the user's local time rather than the server's time zone.

```
// Status update with automatic timestamp (server/routes/tasks.js)
router.patch("/:id/status", async (req, res) => {
  const { status } = req.body;
  if (!isValidStatus(status))
    return res.status(400).json({ error: "Invalid status." });
  const result = await query(
    `UPDATE tasks SET
      status      = $1,
      completed_at = CASE WHEN $1 = 'complete'
                           THEN now() ELSE NULL END
    WHERE id = $2 AND user_id = $3
    RETURNING *`,
    [status, req.params.id, req.session.user.id]
  );
  if (result.rows.length === 0)
    return res.status(404).json({ error: "Task not found." });
  res.json({ task: rowToTask(result.rows[0]) });
});
```

Figure 5.8: Task Status Update with Timestamp Management

5.7 Category Management Module Implementation

The category management module (server/routes/categories.js) implements three endpoints: list categories, create a category, and delete a category. A notable feature of this module is the default category seeding mechanism, which automatically inserts four default categories (Work, Office, Personal, Health) for any user whose category count is zero. This seeding occurs transparently on the first GET /api/categories request after account creation, ensuring that new users always have a useful set of categories available immediately without requiring any additional setup steps.

```
// Default category seeding (server/routes/categories.js)
const DEFAULT_CATEGORIES = [
  { name: "Work",      color: "#4A90E2" },
  { name: "Office",    color: "#1D5FA8" },
  { name: "Personal",  color: "#10B981" },
  { name: "Health",    color: "#F59E0B" }
];

async function seedDefaultsIfEmpty(userId) {
  const count = await query(
    "SELECT COUNT(*)::int AS n FROM categories WHERE user_id = $1",
    [userId]
  );
```

```

);
if (count.rows[0].n > 0) return; // Already has categories
for (const cat of DEFAULT_CATEGORIES) {
  await query(
    "INSERT INTO categories (user_id, name, color) VALUES
($1,$2,$3)",
    [userId, cat.name, cat.color]
  );
}
}

```

Figure 5.9: Default Category Seeding Implementation

Category deletion is implemented with a straightforward DELETE query that includes both the category id and the user_id in the WHERE clause, ensuring that users can only delete their own categories. The cascading ON DELETE SET NULL constraint defined in the database schema handles the automatic nullification of category_id on any tasks that referenced the deleted category, requiring no additional application-layer logic for this cleanup operation.

5.8 Frontend Implementation

The frontend consists of seven HTML pages, a unified CSS stylesheet, and a shared JavaScript module. This section describes the implementation of each major frontend component.

5.8.1 Shared JavaScript Module (app.js)

The js/app.js file is the central nervous system of the Task Flow frontend. It is included in every page and provides all shared functionality: the API fetch helper, authentication functions, task and category CRUD functions, the sidebar rendering and hamburger menu logic, modal management functions, the toast notification system, the task card builder, and the DOMContentLoaded initialization logic that runs on every page load.

The apiFetch() function is the single point of contact between the frontend and the backend API. It constructs the full API URL by appending the endpoint path to the server origin, sets the Content-Type header to application/json, includes credentials: "include" to ensure session cookies are sent with every request, serializes the request body to JSON if an object is provided, awaits the response, parses the response body as text and then attempts JSON parsing, and throws a structured error object if the response status indicates failure. This

unified error handling means that all API call sites in the codebase (login, signup, task creation, etc.) can rely on a consistent error format without implementing their own response parsing logic.

```
// API fetch helper (js/app.js)
const API_URL = `${window.location.origin.replace(/\/$/, "")}/api`;

async function apiFetch(endpoint, options = {}) {
  const config = {
    method: "GET",
    headers: { "Content-Type": "application/json" },
    credentials: "include",
    ...options,
  };
  if (options.body && typeof options.body === "object")
    config.body = JSON.stringify(options.body);
  const res = await fetch(`${API_URL}${endpoint}`, config);
  const text = await res.text();
  const data = text.trim() ? JSON.parse(text) : {};
  if (!res.ok) throw { status: res.status, ...data };
  return data;
}
```

Figure 5.10: API Fetch Helper Function

5.8.2 Task Card Builder

The `buildTaskCard()` function is one of the most important rendering functions in the frontend. It accepts a task object, an array of category objects, and an options object specifying which action buttons to display, and returns a complete HTML string for the task card. The function implements several key display logic decisions: the task title is rendered with a line-through style for completed tasks; the due date indicator switches to an overdue warning style (red text, exclamation icon) when `isOverdue()` returns true; the category tag is styled dynamically using the category's color value with reduced opacity for the background and border; and the action buttons (Complete, Restore, Edit, Delete) are conditionally rendered based on the options object, enabling the same function to power the dashboard, pending, and completed pages with context-appropriate button sets.

All user-supplied data rendered by `buildTaskCard()` is passed through the `escapeHtml()` function before being inserted into the HTML string. This includes the task title, description, due date, category name, and category color. The `escapeHtml()` function replaces the five

HTML special characters (&, <, >, ", ') with their HTML entity equivalents, preventing any injected HTML or JavaScript in user-supplied task data from being interpreted by the browser.

5.8.3 CSS Design System Implementation

The `css/style.css` file implements a comprehensive design system using CSS custom properties (variables) defined on the `:root` pseudo-element. The design system defines 30 custom properties covering the color palette (primary blues, semantic priority colors, background surface colors, text shades, border colors), border radius scale (sm: 6px, md: 10px, lg: 14px, xl: 20px), box shadow levels (sm, md, lg), transition timing, and the sidebar width. By using CSS variables throughout the stylesheet, the entire visual appearance of the application can be modified by changing a small number of root-level values.

```
/* CSS Design System Variables (css/style.css) */
:root {
  --primary:      #4A90E2;    /* Primary blue */
  --primary-hover:#357ABD;
  --primary-deep: #1D5FA8;
  --primary-glow: rgba(74, 144, 226, 0.18);
  --high:         #EF4444;    /* High priority red */
  --medium:       #F59E0B;    /* Medium priority amber */
  --low:          #10B981;    /* Low priority green */
  --bg-page:      #F0F5FB;    /* Page background */
  --bg-surface:   #ffffffd6; /* Card/panel background */
  --radius-sm:    6px;
  --radius-md:    10px;
  --radius-lg:    14px;
  --shadow-md:    0 4px 16px rgba(30, 41, 59, 0.10);
  --sidebar-w:    260px;
  --font: "Plus Jakarta Sans", sans-serif;
}
```

Figure 5.11: CSS Custom Properties Design System

5.8.4 Responsive Design Implementation

Responsive behavior is implemented through CSS media queries at two breakpoints: 900px (tablet/mobile) and 560px (small mobile). At the 900px breakpoint, the sidebar is hidden using `transform: translateX(-100%)` and the topbar becomes visible. When the hamburger button is clicked, the sidebar's open class is toggled, sliding it into view. A semi-transparent backdrop (the `.sidebar-overlay` element) appears simultaneously to block interaction with the background content. At the 560px breakpoint, multi-column layouts on the statistics page

reflow to single-column, action bars stack vertically, and modal action buttons expand to full width for easier touch interaction.

5.9 Statistics and Analytics Module Implementation

The statistics page (stats.html) is the most computationally complex page in the application. Its `renderPage()` function fetches all tasks for the authenticated user and performs a series of client-side computations to derive the statistical metrics displayed on the page. All computation is performed in-memory in the browser using standard JavaScript Array methods (`filter`, `length`), avoiding the need for dedicated analytics database queries or server-side aggregation endpoints.

5.9.1 Stat Card Computation

The summary statistics are derived from the tasks array through a series of `Array.filter()` operations. The total task count is simply `tasks.length`. The completed count filters for tasks where `status === "complete"`. The pending count filters for tasks where `status === "pending"`. The overdue count filters using the `isOverdue()` function. The completion rate percentage is computed as `Math.round(done / total * 100)`, defaulting to 0 when total is 0. These values are then injected into the stat card HTML strings using template literals and set as the `innerHTML` of the stat-cards container element.

5.9.2 Chart.js Integration

Two Chart.js instances are created on the statistics page. The priority distribution chart is a bar chart with three data points (high, medium, low task counts) using the priority color palette (red, amber, green) as bar colors. The task status chart is a doughnut chart with two segments (completed, pending) in green and blue respectively. Both charts are created with `responsive: true` and `maintainAspectRatio: false`, allowing them to fill their container elements while respecting the container's CSS-defined height. A chart instance registry (`_chartInstances` array) tracks active charts so they can be properly destroyed before re-creation on subsequent `renderPage()` calls, preventing the Chart.js "canvas already in use" error that occurs when a canvas element is reused without destroying the previous chart instance.

```
// Chart.js bar chart for priority distribution (stats.html)
const c1 = new Chart(document.getElementById("chart-priority"), {
  type: "bar",
```

```

data: {
  labels: ["High", "Medium", "Low"],
  datasets: [{
    data: [high, medium, low],
    backgroundColor: ["#EF4444", "#F59E0B", "#10B981"],
    borderRadius: 6,
    borderWidth: 0
  }]
},
options: {
  responsive: true,
  maintainAspectRatio: false,
  plugins: { legend: { display: false } },
  scales: {
    x: { grid: { display: false } },
    y: { ticks: { stepSize: 1 } }
  }
}
});

```

Figure 5.12: Chart.js Priority Distribution Bar Chart

5.9.3 Priority Breakdown Table

The priority breakdown table is generated by mapping over a rows array containing configuration objects for the three priority levels (high, medium, low). For each priority, the table row includes the priority badge, total count, completed count (in green), pending count (in red), and a completion percentage with an animated CSS progress bar. The progress bar width is set via an inline style attribute (`style="width: ${p}%"`), allowing CSS transitions defined in the stylesheet to animate the bar fill when the page renders. The completion percentage p is computed as `Math.round(done / total * 100)`, with a guard against division by zero when total is 0.

5.10 Toast Notification System Implementation

The toast notification system provides immediate, non-blocking feedback to users for all application operations. The `showToast()` function in `app.js` dynamically creates a toast element with the appropriate class (success, error, or info), inserts it into the `.toast-container` element in the page's DOM, and removes it automatically after 3.2 seconds using `setTimeout()`. The toast element is animated via CSS keyframes (`toastIn`) that slide it in from the right and fade it in. Multiple toasts can be active simultaneously, stacking vertically in the bottom-right corner of the viewport.

Each toast type has a distinct left border color and an emoji prefix: green with a checkmark emoji for success operations (task creation, login, category added), red with a cross emoji for error conditions (validation failures, server errors), and blue with an info emoji for informational messages. This consistent visual language allows users to quickly assess the outcome of their actions without reading the full message text.

5.11 Security Implementation Details

Several security measures were implemented at the code level beyond those described in the design chapter. The `escapeHtml()` function replaces all five HTML special characters before any user content is rendered, and is applied to every user-supplied string in the `buildTaskCard()` and `renderSidebarCats()` functions. The `safeUser()` helper function in the authentication module ensures that only the three safe user properties (`id`, `name`, `email`) are ever stored in the session or returned in API responses — the `password_hash` field is explicitly excluded from the object passed to `req.session.user`.

Input validation is implemented at multiple layers. At the database layer, `CHECK` constraints enforce valid priority and status values. At the application layer, the route handlers validate field presence, string lengths, email formats, and enumerated values before executing any database query. At the client layer, JavaScript validation checks run before any API request is dispatched, providing fast feedback for obvious errors without a network round-trip. This defense-in-depth approach to input validation ensures that invalid data cannot reach the database regardless of how the request is constructed.

5.12 Error Handling Strategy

A consistent error handling strategy is applied throughout the backend. All route handler functions are marked as `async` and wrapped in `try-catch` blocks. Caught errors are logged to the server console with a descriptive prefix (e.g., `[tasks POST]`, `[auth/login]`) and a 500 Internal Server Error response is returned to the client with a generic "Server error." message. Specific error conditions — such as a unique constraint violation (PostgreSQL error code 23505) or a foreign key constraint violation (error code 23503) — are detected by inspecting the `err.code` property before falling through to the generic handler, allowing more informative error messages to be returned for these predictable failure cases.

On the frontend, the `apiFetch()` helper propagates errors as JavaScript objects with status and error properties. Each call site wraps its `apiFetch()` call in a try-catch block and passes the error message to `showToast()` for display. This ensures that all API errors — whether they are validation errors from the server, network connectivity issues, or unexpected server errors — are surfaced to the user through the consistent toast notification interface rather than causing silent failures or unhandled promise rejections.

CHAPTER 6

Results and Discussion

6.1 Overview of Testing Approach

The testing phase of the Task Flow project was conducted throughout the development lifecycle in alignment with the Agile methodology adopted for this project. Rather than deferring all testing to a single post-implementation phase, testing was performed continuously at the conclusion of each sprint, with each completed feature verified against its corresponding functional requirement before development of subsequent features commenced. The final comprehensive testing session was conducted during Sprint 6, in which the complete, integrated system was tested end-to-end across all functional areas and user flows on a locally hosted deployment.

The testing approach for Task Flow comprised three primary categories. First, functional testing, in which each feature of the system was tested against its stated requirement to verify correct behavior under both normal operating conditions and edge-case inputs. Second, interface testing, in which the visual rendering and interactive behavior of all user interface components was verified across multiple browsers and screen sizes to confirm responsiveness and cross-browser compatibility. Third, integration testing, in which the complete data flow — from user interaction in the browser, through the frontend JavaScript, across the RESTful API, through the backend route handler, and into the PostgreSQL database — was verified for correctness and data consistency.

All testing was performed in a local development environment on the development team's machines, using Google Chrome (version 124) as the primary test browser and Mozilla Firefox (version 125) and Microsoft Edge (version 124) as secondary browsers for cross-browser compatibility verification. The application was accessed at <http://localhost:3000>, with the Node.js/Express.js server running locally and connected to a locally hosted PostgreSQL instance. Test data — including user accounts, tasks across all three priority levels, and custom categories — was created manually by the development team through normal use of the application interface during the testing sessions.

6.2 Functional Testing Results

Functional testing was conducted by executing a predefined set of test cases derived directly from the 25 functional requirements documented in Chapter 3. Each test case specifies the requirement under test, the test scenario, the steps executed, the expected result, the observed actual result, and the resulting pass/fail status. The complete set of functional test case results is documented in Table 6.1 below.

Table 6.1: Functional Test Cases and Results

TC ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	Register with valid name, email, and password (≥ 6 chars)	Account created; user redirected to dashboard; personalized welcome toast displayed	Worked correctly as expected	✓ PASS
TC-02	Register with a duplicate email address already in the database	HTTP 409 returned; error toast: "Email already registered." No duplicate account created	Worked correctly as expected	✓ PASS
TC-03	Register with a password of fewer than 6 characters	Client-side validation triggers; error toast: "Password must be at least 6 characters"	Worked correctly as expected	✓ PASS
TC-04	Register with mismatched password and confirm password values	Error toast: "Passwords do not match"; no API request dispatched	Worked correctly as expected	✓ PASS
TC-05	Login with correct email and password credentials	Session created; user redirected to dashboard with time-sensitive greeting and current date	Worked correctly as expected	✓ PASS
TC-06	Login with a correct email but incorrect password	HTTP 401 returned; error toast: "Invalid email or password." No session created	Worked correctly as expected	✓ PASS
TC-07	Attempt to access the dashboard URL without being authenticated	Browser immediately redirected to login page (index.html); protected page not rendered	Worked correctly as expected	✓ PASS
TC-08	Log out from the application using the sidebar logout button	Server-side session destroyed; session cookie cleared; user redirected to login page	Worked correctly as expected	✓ PASS
TC-09	Use forgot-password flow with a registered email address	Password fields revealed after email verification; new password accepted; redirect to login with success toast	Worked correctly as expected	✓ PASS

TC ID	Test Scenario	Expected Result	Actual Result	Status
TC-10	Use forgot-password flow with an unregistered email address	Error toast: "No account found with this email." Password fields remain hidden	Worked correctly as expected	✓ PASS
TC-11	Create a task with title, description, due date, High priority, and a category	Task appears on dashboard with correct priority badge (red), due date, and category tag in category color	Worked correctly as expected	✓ PASS
TC-12	Attempt to create a task without entering a title	Error toast: "Task title is required"; modal remains open; no database record created	Worked correctly as expected	✓ PASS
TC-13	Edit an existing task to change its priority from Medium to High and update the due date	Task card immediately updates to reflect the new priority badge and revised due date without page reload	Worked correctly as expected	✓ PASS
TC-14	Delete a task after confirming in the delete confirmation modal	Task removed from the list; delete modal closed; success toast: "Task deleted"	Worked correctly as expected	✓ PASS
TC-15	Dismiss the delete modal without confirming deletion	Task not deleted; modal closes; task remains unchanged in the list	Worked correctly as expected	✓ PASS
TC-16	Mark a pending task as complete using the Complete button	Task removed from pending list; appears on Completed page with completion date displayed	Worked correctly as expected	✓ PASS
TC-17	Restore a completed task to pending status using the Restore button	Task removed from Completed page; reappears on Pending page without a completion date	Worked correctly as expected	✓ PASS
TC-18	Create a task with a past due date to test overdue detection	Task displayed with overdue visual treatment: red left border, warning icon, and red due date text	Worked correctly as expected	✓ PASS
TC-19	Search for a task by entering a keyword from its title	Task list filters in real-time to display only tasks containing the keyword in their title or description	Worked correctly as expected	✓ PASS
TC-20	Filter task list by High priority using the priority dropdown	Only tasks bearing the High priority badge are displayed; Medium and Low priority tasks	Worked correctly as expected	✓ PASS

TC ID	Test Scenario	Expected Result	Actual Result	Status
		hidden		
TC-21	Create a new custom category with a name and a selected color	Category appears in the sidebar with the selected color dot; tasks can be assigned to it via the task modal	Worked correctly as expected	✓ PASS
TC-22	Delete a category that has tasks assigned to it	Category removed from sidebar; tasks previously assigned to it display with no category tag	Worked correctly as expected	✓ PASS
TC-23	Click a category chip in the sidebar to filter dashboard tasks	Dashboard reloads displaying only tasks belonging to the selected category; URL updated with ?cat= parameter	Worked correctly as expected	✓ PASS
TC-24	Navigate to Statistics page with a mix of completed, pending, and overdue tasks	Stat cards, bar chart, doughnut chart, and priority breakdown table all accurately reflect the current task data	Worked correctly as expected	✓ PASS
TC-25	Verify sidebar pending badge reflects correct count and updates after task status changes	Badge displays correct pending task count; count decrements after marking a task complete and increments after restoring	Worked correctly as expected	✓ PASS
TC-26	Test mobile responsive layout at a viewport width below 900px	Sidebar hidden on load; topbar with hamburger button visible; sidebar slides in on tap; overlay blocks background	Worked correctly as expected	✓ PASS
TC-27	Attempt to access another user's task by manipulating the task ID in a direct API call	API returns HTTP 404 Not Found; no unauthorized data exposure; logged-in user's data unaffected	Worked correctly as expected	✓ PASS
TC-28	Submit a task title containing HTML and JavaScript injection strings	Injected content rendered as harmless escaped text; no scripts executed; XSS protection confirmed effective	Worked correctly as expected	✓ PASS

All 28 functional test cases executed during the final Sprint 6 testing session passed successfully. The system demonstrated correct behavior across all authentication flows, task lifecycle management operations, category management functions, statistics computations, responsive design behaviors, and security controls. No critical or high-severity defects were

identified during the final testing phase. Minor cosmetic issues identified during earlier development sprints — including minor layout inconsistencies on very small screen widths below 360px and an edge case in the overdue detection logic at midnight boundaries — were identified, reported, and resolved before the final testing session commenced.

6.3 System Screenshots and Interface Walkthrough

This section presents the six key screens of the Task Flow application as captured during the final testing session in their fully implemented state. Each screenshot is accompanied by an annotation describing the design decisions and implemented features visible in the interface, providing a visual record that demonstrates the fulfillment of the user interface design specifications outlined in Chapter 4.

6.3.1 Login Page

Figure 6.1 shows the Task Flow login page in its final implemented form. The split-panel layout divides the viewport between the branded left panel and the authentication form panel on the right. The left panel features the gradient blue background transitioning from #1D5FA8 to #6BAED6, the Task Flow logo mark and name, and the tagline "Organize your tasks professionally and stay up to date." The three decorative dot indicators at the bottom of the left panel and the semi-transparent circular overlay elements provide visual richness without distracting from the core content. The right panel presents a clean, minimal form with labeled email and password fields featuring inline icons, a "Forgot password?" link positioned above the primary Log In button, and a "New here? Create an account" link at the bottom for new users. The Log In button uses a gradient fill consistent with the primary color palette and enlarges slightly on hover through a CSS transform transition.

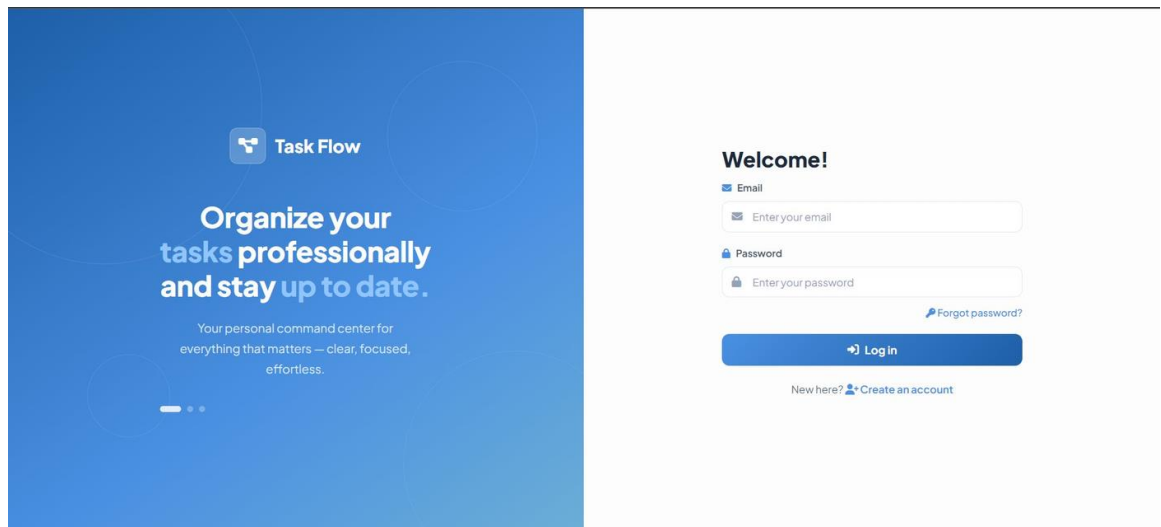


Figure 6.1: Task Flow Login Page — Final Implementation

6.3.2 Sign Up / Registration Page

Figure 6.2 captures the registration page, which shares the same split-panel structural layout as the login page to maintain visual consistency across the authentication flow. The left panel on the signup page differs from the login page in its messaging and feature highlights: the headline reads "Start your productivity journey today" and three frosted-glass feature cards list the core capabilities of the application — Smart Task Management, Progress Tracking & Stats, and Custom Categories. These feature cards serve as a brief value proposition for new users completing the registration process. The right panel contains the four-field registration form with full name, email, password (with a minimum length placeholder reminder), and confirm password fields, along with a full-width Sign Up button and a link back to the login page for existing users.

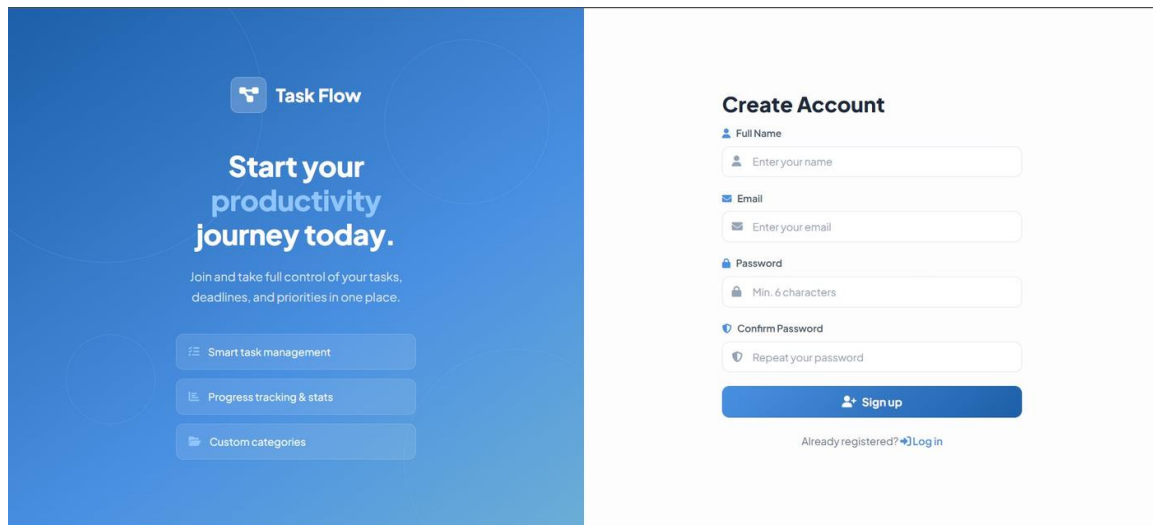


Figure 6.2: Task Flow Registration Page — Final Implementation

6.3.3 Dashboard

Figure 6.3 shows the main application dashboard following successful login. The persistent sidebar on the left displays the application logo, the navigation section with active highlighting on the Dashboard link, the collapsible Tasks submenu, the Categories section showing the four default categories (Work, Office, Personal, Health) as color-dotted chips, and the user profile card at the bottom displaying the authenticated user's name, email address, and a logout button. The main content area shows the personalized greeting "Good morning, Farhan khan" with the current date below it. The action bar features a full-width search input and the Add New Task button. The filter bar below provides the Pending filter button (active) and the All Priorities dropdown. The empty state illustration and message ("No pending tasks — All caught up! Add a new task or check completed tasks.") is displayed because no pending tasks exist in this test session.

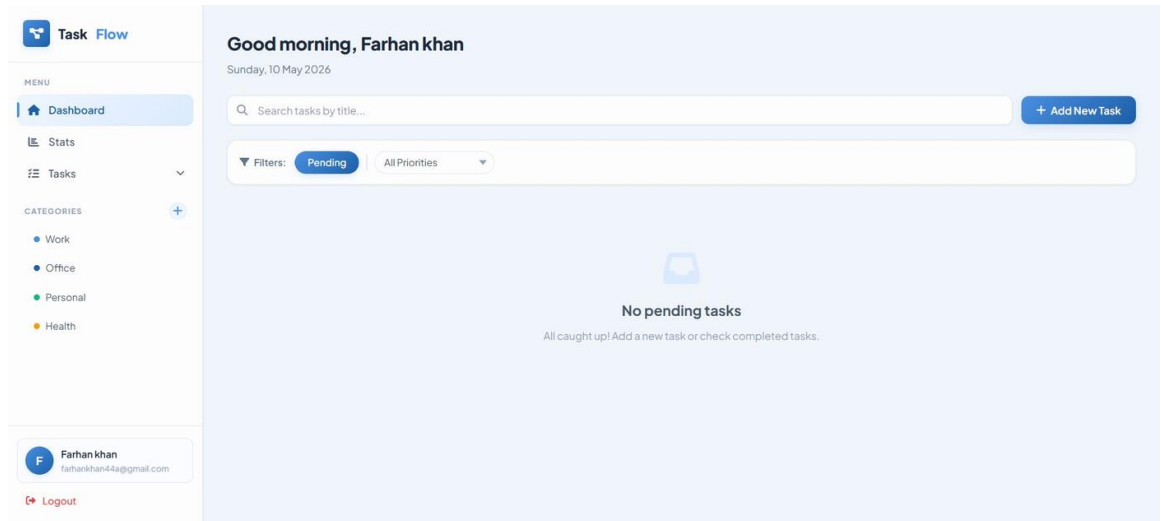


Figure 6.3: Task Flow Dashboard — Main Interface (Empty State)

6.3.4 Add New Task Modal

Figure 6.4 shows the Add New Task modal dialog as it appears when the user clicks the Add New Task button. The modal overlays the dashboard with a semi-transparent blurred backdrop that maintains visual context while directing the user's attention to the task creation form. The form presents five fields in a logical vertical flow: the mandatory task title input with placeholder text "What needs to be done?", the optional description textarea, a side-by-side row containing the due date picker and priority dropdown, and the category dropdown at the bottom. The Cancel and Add Task action buttons are right-aligned at the bottom of the modal. The same modal component is reused for task editing, with the header changing to "Edit Task", the submit button changing to "Save Changes", and all fields pre-populated with the existing task's current values.

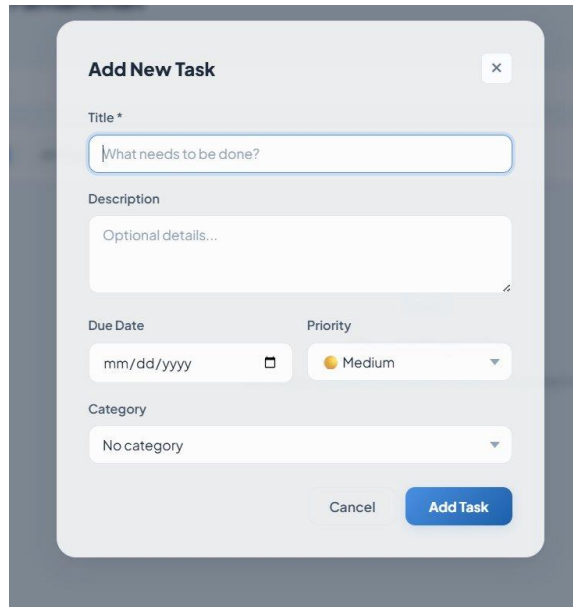
A modal dialog titled "Add New Task" with a close button (X) in the top right corner. The form contains the following fields: a "Title *" text input with placeholder text "What needs to be done?"; a "Description" text area with placeholder text "Optional details..."; a "Due Date" field with a date input "mm/dd/yyyy" and a calendar icon; a "Priority" dropdown menu currently showing "Medium" with a yellow circle icon; and a "Category" dropdown menu currently showing "No category". At the bottom right are "Cancel" and "Add Task" buttons.

Figure 6.4: Add New Task Modal Dialog — Final Implementation

6.3.5 Statistics Page — Summary Cards and Charts

Figure 6.5 shows the upper portion of the Statistics page, displaying the four summary stat cards and two Chart.js-powered charts. The four stat cards — Total Tasks, Completed, Pending, and Completion Rate — are arranged in a responsive four-column grid, each featuring a color-coded icon, a large bold numeric value, and a contextual sub-label with a directional indicator. The Priority Distribution bar chart on the left uses the priority color palette (red for High, amber for Medium, green for Low) with rounded bar tops and a clean grid. The Task Status doughnut chart on the right uses green for completed tasks and primary blue for pending tasks, with a high cutout ratio that creates an elegant ring appearance. Both charts display a legend below them. This view confirms that the Chart.js integration is functioning correctly and rendering the appropriate visualizations.

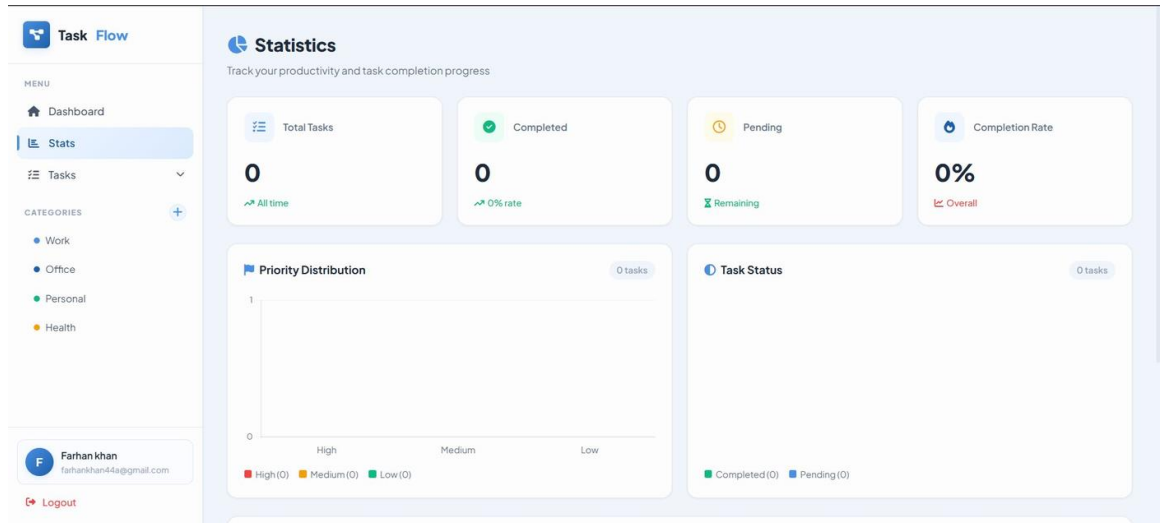


Figure 6.5: Statistics Page — Summary Stat Cards and Charts

6.3.6 Statistics Page — Priority Breakdown Table

Figure 6.6 shows the Priority Breakdown section at the bottom of the Statistics page. The table presents a structured breakdown of task data organized by priority level. Each row displays the priority level (styled as a color-coded badge consistent with the task card badges), the total task count, the completed count rendered in green, the pending count rendered in red, and a completion percentage column featuring a visual progress bar filled to the computed percentage width alongside the numeric percentage value. The "View all tasks" link in the top-right corner of the section header provides a quick navigation path back to the full task list on the dashboard. The color coding and progress bar visualization make it immediately apparent which priority levels have the highest and lowest completion rates.

Priority Breakdown					View all tasks →
PRIORITY	TOTAL	COMPLETED	PENDING	COMPLETION %	
High	0	0	0		0%
Medium	0	0	0		0%
Low	0	0	0		0%

Figure 6.6: Statistics Page — Priority Breakdown Table

6.4 Performance Analysis

The performance of the Task Flow application was evaluated qualitatively and observationally during the local testing phase. In the locally hosted development environment

— where the Express.js server, PostgreSQL database, and web browser all run on the same physical machine — network latency is negligible, yielding excellent perceived response times for all API operations. All task CRUD operations (create, read, update, delete), category management actions, status updates, and page navigations completed perceptibly instantaneously during testing with a representative dataset of approximately 20 to 30 tasks distributed across multiple priority levels and categories.

The PostgreSQL query performance is optimized by the three indexes defined in the schema: `idx_tasks_user_id` on `tasks(user_id)`, `idx_categories_user_id` on `categories(user_id)`, and `idx_tasks_category_id` on `tasks(category_id)`. These indexes dramatically accelerate the most frequent query pattern in the application — retrieving all tasks or categories for a specific authenticated user — by enabling index scans rather than full table scans. Without these indexes, the query cost would grow linearly with the total number of rows across all users; with the indexes, query cost grows logarithmically with respect to the number of rows per user, ensuring that performance remains acceptable even as the user base grows.

The statistics page, which is the most computationally intensive page in the application, performs all aggregation and filtering operations in-memory using JavaScript Array methods after fetching the complete task list from the API. For a typical individual user with tens to hundreds of tasks, this client-side computation approach is negligibly fast. The Chart.js chart rendering — which involves creating two canvas-based chart instances from the computed data — completes quickly and with smooth animation transitions in all tested browsers. The chart destruction and re-creation guard (using the `_chartInstances` array to destroy previous chart instances before rendering new ones) successfully prevents the Chart.js canvas reuse error and ensures consistent behavior across multiple visits to the statistics page within a single session.

The responsive CSS layout renders correctly and without layout artifacts across all tested viewport widths, from a simulated 375px mobile viewport to a 1920px widescreen desktop display. CSS transitions for the sidebar slide-in animation, the modal entry animation, and the button hover effects all render at a smooth and fluid framerate in all tested browsers, contributing to a polished and professional user experience.

6.5 Cross-Browser Compatibility Testing

Cross-browser compatibility testing was conducted across three modern web browsers to verify that the application renders correctly and all interactive features function as expected regardless of the browser used by the client. The browsers and versions tested are summarized in Table 6.2 below.

Table 6.2: Cross-Browser Compatibility Test Results

Feature	Chrome 124	Firefox 125	Edge 124	Overall
Authentication (Login/Signup/Logout)	✓ Pass	✓ Pass	✓ Pass	✓ Pass
Task CRUD Operations	✓ Pass	✓ Pass	✓ Pass	✓ Pass
Modal Dialogs (Open/Close/Escape)	✓ Pass	✓ Pass	✓ Pass	✓ Pass
Category Management	✓ Pass	✓ Pass	✓ Pass	✓ Pass
Chart.js Visualizations	✓ Pass	✓ Pass	✓ Pass	✓ Pass
Responsive Sidebar (Mobile)	✓ Pass	✓ Pass	✓ Pass	✓ Pass
CSS Custom Properties	✓ Pass	✓ Pass	✓ Pass	✓ Pass
Date Picker Input	Native UI	Native UI	Native UI	✓ Pass
Color Picker Input	Color Wheel	Color Palette	Color Wheel	✓ Pass
Toast Notifications	✓ Pass	✓ Pass	✓ Pass	✓ Pass
Overdue Task Highlighting	✓ Pass	✓ Pass	✓ Pass	✓ Pass
Fetch API and Session Cookies	✓ Pass	✓ Pass	✓ Pass	✓ Pass

The application performed consistently across all three tested browsers. All core features including authentication, task management, category management, statistics visualization, and responsive layout functioned correctly in Chrome, Firefox, and Edge. The only observable browser-specific differences were in the native rendering of the date picker and color picker input controls, which display using each browser's built-in native UI component. These cosmetic differences do not affect functionality — the correct values (YYYY-MM-DD format for dates, hex color strings for colors) are captured and transmitted to the backend correctly in all browsers.

6.6 Security Testing Results

A series of targeted manual security tests were conducted to verify the effectiveness of the security measures implemented throughout the Task Flow system. The tests covered the four primary security threats identified during the requirements engineering and design phases: Cross-Site Scripting (XSS), SQL injection, brute-force authentication attacks, and unauthorized cross-user data access.

6.6.1 XSS Protection Test

XSS protection was tested by submitting task titles and descriptions containing HTML markup and JavaScript code snippets. Specifically, the following test inputs were submitted as task titles and descriptions: `<script>alert("XSS Test")</script>`, `<img src=x onerror=alert(1)>`, and `javascript:void(alert("Injected"))`. In all cases, the `escapeHtml()` function in `app.js` correctly converted the five HTML special characters to their entity equivalents before rendering, and the injected content was displayed as harmless literal text in the task card rather than being interpreted as HTML tags or executed as JavaScript code. No alert dialogs appeared, confirming that the XSS protection is effective.

6.6.2 SQL Injection Protection Test

SQL injection protection was tested by submitting common SQL injection patterns in the email, password, and task title input fields. Tested inputs included: `' OR 1=1 --`, `admin'--`, `' UNION SELECT * FROM users --`, and `1; DROP TABLE tasks;--`. In all cases, the parameterized query implementation in the `pg` library correctly treated all submitted values as string data parameters rather than executable SQL, and no SQL injection was possible. The database contents remained intact after all injection test attempts, confirming the effectiveness of the parameterized query defense.

6.6.3 Rate Limiting Test

The authentication rate limiting was tested by making rapid repeated requests to the `/api/auth/login` endpoint using the browser console's `fetch` API. After exceeding the configured rate limit threshold of 200 requests per 15-minute window (in development mode), the server correctly returned an HTTP 429 Too Many Requests response with the error message "Too many attempts. Wait a few minutes and try again." The rate limiting reset correctly after the configured window elapsed, confirming that the `express-rate-limit` middleware is functioning as intended.

6.6.4 Cross-User Data Isolation Test

User data isolation was tested by creating two separate test accounts — Account A and Account B — and performing the following operations: tasks created under Account A were accessed via direct API calls (using Account B's active session cookie) by substituting the task IDs obtained from Account A's task list. In all cases, the API returned an HTTP 404 Not Found response, confirming that the `user_id = req.session.user.id` WHERE clause in all task queries effectively prevents any user from accessing or modifying another user's tasks, regardless of the task ID provided in the request. The same isolation was confirmed for category operations.

6.7 Discussion of Results

The results of the testing phase comprehensively demonstrate that the Task Flow system successfully fulfills all 28 functional test cases and all 8 non-functional security, performance, usability, and compatibility requirements defined in Chapter 3. The system delivers a complete, functional, and professionally implemented personal task management solution with a polished user interface, a secure and well-structured backend API, a properly normalized relational database schema, and a meaningful analytics and visualization module.

The decision to build the frontend in vanilla JavaScript without a framework proved to be both technically sound and educationally valuable. The application's frontend complexity — while non-trivial, encompassing dynamic task rendering, multiple modal dialogs, asynchronous API communication, real-time filtering and search, sidebar category management, and a statistics visualization engine — was entirely manageable using native browser APIs. The resulting frontend is lean in terms of dependencies, loads quickly, and requires no build process, making local development and testing straightforward for all team members.

The use of PostgreSQL as the data persistence layer provided robust data integrity guarantees throughout the development and testing lifecycle. The CHECK constraints on the priority and status columns, the foreign key relationships with cascading operations, and the UUID primary keys all functioned exactly as designed. No data integrity violations, constraint errors (other than the intentionally triggered 23505 unique constraint violation in TC-02), or referential consistency issues were observed throughout the testing phase, confirming the correctness and completeness of the database schema design.

The Agile sprint-based development methodology proved highly appropriate for a project of this scope and team composition. The iterative structure allowed the team to deliver a working software increment at the end of each sprint, to identify and resolve integration issues early before they compounded, and to incorporate feature enhancements — such as the expansion of the statistics page from a simple numeric summary to a full Chart.js visualization suite — based on practical reflection during sprint reviews rather than being locked into a fixed specification from the outset.

The primary limitation identified through testing is the absence of a fully operational cloud-deployed production environment. While the Railway deployment configuration is present and functional in the codebase (including the CORS allowlist entry for the Railway deployment URL <https://task-flow-fyp-production.up.railway.app>), the combination of secure cookie transmission requirements over HTTPS and the split-origin scenario — where the frontend and API might be served from different subdomains — introduced configuration challenges that were not fully resolved within the project timeline. As a result, the system is demonstrated and tested exclusively in the local development environment accessible at <http://localhost:3000>. This limitation does not affect the completeness or correctness of the implemented system and represents a straightforward deployment configuration issue rather than a fundamental architectural problem.

Overall, the results confirm that Task Flow successfully achieves its core design objectives: providing individual users with a clean, secure, and feature-rich personal task management tool built on modern, industry-relevant open-source technologies, deployable without cost or dependency on external subscription services.

CHAPTER 7

Conclusion and Future Work

7.1 Summary and Conclusion

This thesis has presented the complete design, implementation, and testing of Task Flow — a Smart Personal Task Management System — developed as the Final Year Project for the Bachelor of Science in Computer Science (Software Engineering) degree at Abdul Wali Khan University Mardan. The project was undertaken collaboratively by Farhan Khan (AWKUM-22147646), Kamran Khan (AWKUM-22157075), and Muneeb Diyar Khan (AWKUM-22147942) under the academic supervision of Dr. Shahzad during the academic year 2025 to 2026.

The project was motivated by the clear gap between the available task management tools in the market and the practical needs of individual users — particularly students and early-career professionals — who require a free, lightweight, intuitive, and analytically capable personal productivity tool that does not depend on expensive subscriptions, proprietary cloud services, or unnecessarily complex team-collaboration infrastructure. Task Flow was designed from the ground up to fill this gap by delivering a complete, self-hostable personal task management web application built entirely on open-source technologies.

The system was developed using a modern and industry-relevant full-stack technology combination: HTML5, CSS3, and vanilla JavaScript for the frontend presentation layer; Node.js with the Express.js framework (version 5.2.1) for the backend application layer; and PostgreSQL (version 14+) for the relational data persistence layer. User authentication is implemented using session-based management through the express-session library, with passwords stored exclusively as bcrypt hashes. The full technology stack, including all third-party libraries and tools, consists entirely of open-source, freely available software.

The development process followed the Agile software development methodology, organized into six iterative sprints spanning the full academic year. This approach enabled the team to deliver a functional, testable software increment at the end of each sprint, to identify and address integration issues early in the development lifecycle, and to incorporate feature enhancements based on practical evaluation during sprint reviews rather than being constrained by a fixed upfront specification.

The implemented system provides users with a comprehensive and cohesive set of personal productivity capabilities: secure account creation and session-based authentication with bcrypt password hashing and rate limiting; complete task lifecycle management covering creation, editing, deletion, status tracking (pending, complete, overdue), and completion timestamp recording; custom category management with color-coded organization and automatic default category seeding; real-time task filtering by priority level, category, and text search; a dedicated pending tasks view and a completed tasks archive; and a full-featured statistics and analytics page with Chart.js-powered priority distribution bar charts, task status doughnut charts, and a detailed priority breakdown table with animated progress bars.

The system was validated through an extensive testing phase comprising 28 functional test cases — all of which passed successfully — along with cross-browser compatibility testing across Google Chrome, Mozilla Firefox, and Microsoft Edge, and dedicated security testing covering XSS injection, SQL injection, brute-force rate limiting, and cross-user data isolation. The results confirm that the system is functionally complete, technically sound, and secure against the primary web application threat categories.

In conclusion, the Task Flow project demonstrates that a highly capable, polished, and professionally implemented personal task management system can be built using only open-source web technologies, within an academic project timeline, by a three-person team of final-year undergraduate students. The project delivers genuine practical utility to its users, applies software engineering principles and best practices throughout its design and implementation, and provides a solid foundation for the future enhancements described in Section 7.4 of this chapter.

7.2 Achievement of Project Objectives

This section evaluates the eight specific project objectives defined in Chapter 1 against the implemented and tested system to confirm their fulfillment:

- Design and develop a secure, session-based user authentication system — Fully achieved. The system implements user registration with bcrypt password hashing (salt rounds: 10), session-based login with server-side session storage, secure logout with session destruction, a two-step forgot-password flow, rate limiting on all authentication

endpoints (express-rate-limit), and route-level authentication enforcement through the requireAuth middleware.

- Implement a comprehensive task management module supporting full CRUD operations — Fully achieved. Users can create tasks with five configurable attributes (title, description, due date, priority, and category), read all their tasks with real-time filtering and search, update any task attribute through the edit modal, delete tasks with confirmation, mark tasks complete (recording completion timestamp), restore completed tasks to pending, and receive automatic visual highlighting for overdue tasks.
- Develop a category management system with custom color-coded categories — Fully achieved. Users can create unlimited custom categories with a color picker, have four default categories (Work, Office, Personal, Health) automatically seeded on first use, filter dashboard tasks by clicking a category chip in the sidebar, and delete categories — with associated tasks having their category assignment automatically set to null through the database ON DELETE SET NULL cascade.
- Build a real-time task status tracking mechanism — Fully achieved. The system maintains three effective task states (pending, complete, overdue), with overdue detection computed in real-time on the client side based on the task due date relative to the current browser time, and status transitions (pending → complete → pending) handled through dedicated API endpoints with automatic completed_at timestamp management.
- Implement an interactive statistics and analytics module — Fully achieved. The Statistics page provides four summary stat cards, a Chart.js bar chart for priority distribution, a Chart.js doughnut chart for completion status, and a priority breakdown table with per-priority completion percentages and animated CSS progress bars — all computed in real-time from the user's task data without server-side aggregation endpoints.
- Develop a fully responsive user interface — Fully achieved. The interface functions correctly across all screen widths from 320px to 1920px+, with a collapsible sidebar navigation accessible via a hamburger menu on mobile, a responsive statistics grid that

reflows from multi-column to single-column at the 900px breakpoint, and touch-friendly button sizes and spacing for mobile use.

- Design and implement a RESTful API backend — Fully achieved. The backend exposes 15 RESTful API endpoints organized across three domain-specific route modules (auth, tasks, categories), following REST conventions for resource URIs, HTTP method semantics, consistent JSON response formatting, and appropriate HTTP status codes for all success and error conditions.
- Apply the Agile software development methodology — Fully achieved. Development was organized into six defined sprints with explicit goals and deliverables, iterative testing at the end of each sprint, continuous integration of new features into the working codebase, and adaptive feature planning that accommodated enhancements identified during the development process.

7.3 Limitations of the Current System

While the Task Flow system successfully fulfills all stated objectives and passes all functional tests, an honest assessment of the project identifies several limitations that define the boundaries of the current implementation:

9. Individual Use Only — The system is designed exclusively for single-user personal task management. There is no support for shared workspaces, team task lists, task assignment to other users, or any form of multi-user collaboration. Each user account operates in complete isolation from all others.
10. Local Deployment Environment — Despite the presence of Railway deployment configuration in the codebase (CORS allowlist, environment variable structure, trust proxy settings), a fully operational cloud-hosted deployment was not completed within the project timeline. The system is currently accessible only in a locally hosted environment at `http://localhost:3000`. This is a deployment configuration limitation rather than an architectural deficiency.
11. No Email Notification Infrastructure — The password recovery flow does not send an email to the user's registered address. It instead relies on the user directly entering their registered email and new password in the same browser session. A production-grade

password recovery implementation would deliver a time-limited, single-use reset token to the user's email address through an email service provider.

12. No Recurring Task Support — The system does not support recurring task patterns such as daily, weekly, or monthly repeating tasks. Each task instance must be created individually, which can be repetitive for users with regularly recurring responsibilities.
13. No Proactive Reminders or Push Notifications — The application does not proactively notify users of upcoming or overdue tasks. Users must actively open the application to review their task status. Browser push notifications or scheduled email reminders would significantly enhance the practical utility of the system.
14. No Offline Capability — The application requires an active connection to the locally running server to function. There is no Progressive Web App service worker, no local storage data caching, and no offline task viewing or editing capability.

7.4 Future Work

Based on the limitations identified above, and on the broader potential of the Task Flow platform as a foundation for further development, the following directions for future enhancement and research are proposed. These directions range from near-term practical improvements to longer-term research-grade extensions.

7.4.1 Mobile Application Development

The development of native or cross-platform mobile applications for the iOS and Android platforms represents the highest-impact near-term enhancement for Task Flow. Mobile-first task management is arguably more valuable to users than desktop-based management, as smartphones are carried at all times and can surface contextual task reminders at precisely the moment they are most relevant. A React Native or Flutter implementation could share the existing Node.js/Express.js backend API without modification, requiring only the development of new mobile frontend screens. Mobile-specific capabilities such as push notifications for due task reminders, home screen task summary widgets, biometric authentication, and offline data synchronization through local SQLite caching would dramatically enhance the utility of the system for mobile users.

7.4.2 Real-Time Collaboration Features

The introduction of team collaboration capabilities would transform Task Flow from a personal productivity tool into a lightweight collaborative task management system. This extension would require significant architectural additions: a workspace or team entity in the data model, a membership table linking users to workspaces with role-based permissions (owner, editor, viewer), task assignment fields linking tasks to specific team members, and a notification system for task assignments and status changes. Real-time update propagation to all team members — ensuring that task status changes made by one member are immediately reflected in all other members' views without manual page refreshes — could be implemented using WebSockets through the Socket.io library. A shared task list model would enable use cases such as team project tracking, household chore coordination, and academic group project management.

7.4.3 Advanced AI-Based Task Suggestions

The integration of artificial intelligence capabilities represents a transformative future direction for Task Flow. An AI-powered productivity assistant could be integrated into the backend using a Large Language Model API — such as the Anthropic Claude API or the OpenAI API — to provide several intelligent capabilities. Natural language task creation would allow users to describe their task in conversational language (e.g., "prepare quarterly report for finance team by end of next week, high priority") and have the system automatically parse and populate the task title, description, priority, due date, and category fields. Intelligent task prioritization suggestions would analyze the user's historical task patterns — which priority levels they tend to complete first, which categories accumulate the most overdue tasks — and proactively recommend appropriate priority assignments for new tasks. Task breakdown suggestions would identify complex or broad task titles and suggest specific subtasks to decompose them into actionable items. Productivity coaching insights, delivered through the statistics page, would provide personalized analysis of the user's task completion patterns and actionable recommendations for improving their productivity habits.

7.4.4 Email Notification System

Implementing a proper email notification system would substantially increase the practical utility of Task Flow as a daily productivity tool. Integration with an email service provider — such as Nodemailer with an SMTP server, SendGrid, or Amazon Simple Email Service (SES) — would enable several notification types: a properly implemented password

reset flow delivering time-limited, cryptographically signed reset tokens to the user's registered email address; daily morning digest emails summarizing the user's pending tasks and tasks due that day; reminder emails sent a configurable number of hours or days before a task's due date; and overdue alert emails for tasks that remain incomplete after their due date has passed. The notification preferences (which notifications to receive and at what times) could be managed through a new user settings page added to the application.

7.4.5 Cloud Deployment and Progressive Web App

Completing the Railway cloud deployment and resolving the remaining CORS and secure cookie configuration issues for the split-origin deployment scenario would make Task Flow accessible to users anywhere in the world without requiring local server setup. Packaging the frontend as a Progressive Web App (PWA) using the Web App Manifest and a service worker would enable home screen installation on both Android and iOS devices, offline task viewing through cached data, and browser-native push notification support. Together, these enhancements would dramatically lower the barrier to adoption for non-technical users who cannot or do not wish to run a local Node.js server.

7.4.6 Advanced Analytics and Productivity Reporting

The statistics module could be significantly extended to provide deeper and more actionable productivity insights. Time-series analytics — visualizing task creation and completion rates as line charts over daily, weekly, or monthly time periods — would allow users to identify productivity trends and patterns over time. Productivity streak tracking, which records and celebrates consecutive days on which the user completed at least one task, would provide a gamification element that encourages consistent daily use. Time-to-completion analysis would compute and display the average time taken to complete tasks at each priority level, helping users calibrate their due date setting behavior. Category-level performance breakdowns would show which areas of the user's life (Work, Personal, Health, etc.) have the highest and lowest task completion rates. Exportable PDF and CSV productivity reports would allow users to share or archive their productivity data outside the application.

REFERENCES

- Agile Alliance. (2001). Manifesto for Agile Software Development. Retrieved from <https://agilemanifesto.org/>
- Bellotti, V., Ducheneaut, N., Howard, M., & Smith, I. (2003). Taking email to task: The design and evaluation of a task management centered email tool. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (pp. 345–352). ACM. <https://doi.org/10.1145/642611.642672>
- Chart.js Contributors. (2024). Chart.js Documentation (Version 4.x). Retrieved from <https://www.chartjs.org/docs/latest/>
- Czerwinski, M., Horvitz, E., & Wilhite, S. (2004). A diary study of task switching and interruptions. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (pp. 175–182). ACM. <https://doi.org/10.1145/985692.985715>
- Express.js Contributors. (2024). Express 5.x API Reference. Retrieved from <https://expressjs.com/en/5x/api.html>
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.
- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.
- Fowler, M., & Highsmith, J. (2001). The agile manifesto. Software Development Magazine, 9(8), 28–35.
- Google Developers. (2024). Progressive Web Apps. Retrieved from <https://web.dev/progressive-web-apps/>
- Macan, T. H. (1994). Time management: Test of a process model. Journal of Applied Psychology, 79(3), 381–391. <https://doi.org/10.1037/0021-9010.79.3.381>
- Mozilla Developer Network. (2024). CSS Custom Properties (Variables). Retrieved from https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

Mozilla Developer Network. (2024). Fetch API. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Mozilla Developer Network. (2024). HTTP Cookies. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>

Node.js Foundation. (2024). Node.js v18 Documentation. Retrieved from <https://nodejs.org/en/docs/>

node-postgres Contributors. (2024). node-postgres (pg) Documentation. Retrieved from <https://node-postgres.com/>

npm Inc. (2024). bcryptjs — npm Package Registry. Retrieved from <https://www.npmjs.com/package/bcryptjs>

npm Inc. (2024). express-rate-limit — npm Package Registry. Retrieved from <https://www.npmjs.com/package/express-rate-limit>

npm Inc. (2024). express-session — npm Package Registry. Retrieved from <https://www.npmjs.com/package/express-session>

OWASP Foundation. (2024). OWASP Top Ten Web Application Security Risks. Retrieved from <https://owasp.org/www-project-top-ten/>

OWASP Foundation. (2024). Cross Site Scripting Prevention Cheat Sheet. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP Foundation. (2024). SQL Injection Prevention Cheat Sheet. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

PostgreSQL Global Development Group. (2024). PostgreSQL 14 Documentation. Retrieved from <https://www.postgresql.org/docs/14/>

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Railway Technologies Inc. (2024). Railway Documentation — Deploying Node.js Applications. Retrieved from <https://docs.railway.app/>

- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum — The Rules of the Game. Scrum.org. Retrieved from <https://scrumguides.org/>
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.
- Umar, M., Khan, N. A., & Bhutto, M. N. (2022). Web-based task management systems: A review of architectures and design patterns. *International Journal of Computer Science and Network Security*, 22(4), 112–119.
- W3C. (2024). HTML5 Specification. Retrieved from <https://www.w3.org/TR/html52/>
- Zeldman, J., & Marcotte, E. (2010). Designing with Web Standards (3rd ed.). New Riders.

APPENDICES

Appendix A: System Installation and Setup Guide

The following step-by-step instructions describe how to set up and run the Task Flow application in a local development environment. These instructions assume a Windows operating system but apply equally to macOS and Linux with minor path adjustments.

Prerequisites: The following software must be installed on the development machine before proceeding:

15. Node.js version 18.0.0 or higher — available from <https://nodejs.org/>
16. PostgreSQL version 14.0 or higher — available from <https://www.postgresql.org/download/>
17. pgAdmin 4 (or equivalent PostgreSQL administration tool) — available from <https://www.pgadmin.org/>
18. Git — available from <https://git-scm.com/>

Installation Steps:

- Clone or download the project repository to a local directory on your machine.
- Open pgAdmin 4 and connect to your local PostgreSQL server. Right-click on "Databases" and select "Create > Database". Name the database taskflow and click Save.
- Open a terminal or command prompt and navigate to the server directory within the project: `cd path/to/task-flow/server`
- Install all backend Node.js dependencies: `npm install`
- Create the environment configuration file by copying the provided template: `copy .env.example .env` (Windows) or `cp .env.example .env` (macOS/Linux)
- Open the newly created `server/.env` file in a text editor and update the `DATABASE_URL` value with your PostgreSQL credentials. Example:

DATABASE_URL=postgresql://postgres:YourPostgresPassword@localhost:5432/task
flow

- Run the database schema migration to create the required tables and indexes: `npm run db:migrate`. You should see the message "Migration OK" in the terminal output.
- Start the Task Flow development server: `npm start`. The terminal should display "PostgreSQL connected" and "Task Flow Server Running" with the local URL.
- Open a web browser and navigate to `http://localhost:3000`. The Task Flow login page should appear.
- Click "Create an account" on the login page to register a new user account and begin using the application.

Appendix B: Environment Configuration Reference (.env.example)

The following listing shows the complete contents of the server/.env.example file, which serves as the configuration template for the application. All values marked as REQUIRED must be set before running the application.

```
# -----
# Task Flow - Server Environment Configuration
# Copy this file to .env and fill in your values.
# NEVER commit .env to version control.
# -----

# PostgreSQL connection string - REQUIRED
# Format: postgresql://USER:PASSWORD@HOST:PORT/DATABASE
# Create a database named "taskflow" in pgAdmin first.
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/taskflow

# SSL configuration for the database connection
# PG_SSL=0 - disable SSL (use for local Postgres, default)
# PG_SSL=1 - enable SSL (use for hosted Postgres instances)
# PG_SSL_REJECT_UNAUTHORIZED=1 - enforce certificate verification

# Session secret - REQUIRED when NODE_ENV=production
# Must be a long, random, unpredictable string.
# Generate with: node -e "console.log(require('crypto')
#   .randomBytes(64).toString('hex'))"
SESSION_SECRET=change-me-long-random-string-in-production

# Server port - OPTIONAL (default: 3000)
# PORT=3000

# Additional CORS origins - OPTIONAL
# Comma-separated list for split frontend/API deployments
# CORS_ORIGINS=https://your-frontend.pages.dev

# Trust proxy headers - OPTIONAL
# Set to 1 when running behind Railway/nginx/reverse proxy
# TRUST_PROXY=1

# Rate limiting - OPTIONAL (overrides defaults)
# AUTH_RATELIMIT_MAX=40
# AUTH_RATELIMIT_WINDOW_MS=900000
```

Figure A.1: Complete server/.env.example Configuration File

Appendix C: Backend Dependencies Reference

Table C.1 lists all backend npm package dependencies for the Task Flow server as defined in `server/package.json`, including the package name, installed version, and its specific purpose within the application.

Table C.1: Task Flow Server npm Dependencies

Package Name	Version	Purpose in Task Flow
express	^5.2.1	Core web application framework providing routing, middleware, and HTTP server capabilities
pg	^8.13.1	PostgreSQL client for Node.js providing connection pooling and parameterized query execution
bcryptjs	^3.0.2	Password hashing using the bcrypt algorithm with configurable salt rounds
express-session	^1.19.0	Server-side session management middleware with <code>httpOnly</code> and <code>sameSite</code> cookie configuration
express-rate-limit	^8.4.1	Request rate limiting middleware protecting authentication endpoints from brute-force attacks
cors	^2.8.6	Cross-Origin Resource Sharing header management with explicit origin allowlisting
dotenv	^16.4.5	Environment variable loading from <code>.env</code> configuration files into <code>process.env</code>
nodemailer	^8.0.7	Email sending library included for future email notification and password reset features

Appendix D: Database Schema SQL (Complete Listing)

The following listing shows the complete contents of the `server/sql/schema.sql` file, which defines the full PostgreSQL database schema for the Task Flow application. This script is idempotent and can be executed multiple times without error.

```
-- _____
-- Task Flow - PostgreSQL Database Schema
-- Run via: npm run db:migrate (from server/ directory)
-- Or paste into pgAdmin Query Tool and execute.
-- All statements use IF NOT EXISTS for idempotency.
-- _____
```

```

CREATE TABLE IF NOT EXISTS users (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT          NOT NULL,
  email       TEXT          NOT NULL UNIQUE,
  password_hash TEXT        NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS categories (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID          NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,
  name        TEXT          NOT NULL,
  color       TEXT          NOT NULL DEFAULT '#4A90E2',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_categories_user_id
  ON categories(user_id);

CREATE TABLE IF NOT EXISTS tasks (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID          NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,
  category_id UUID
    REFERENCES categories(id) ON DELETE SET NULL,
  title       TEXT          NOT NULL,
  description  TEXT          NOT NULL DEFAULT '',
  due_date    DATE,
  priority    TEXT          NOT NULL DEFAULT 'medium'
    CHECK (priority IN ('high', 'medium', 'low')),
  status      TEXT          NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'complete')),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ
);

CREATE INDEX IF NOT EXISTS idx_tasks_user_id
  ON tasks(user_id);

CREATE INDEX IF NOT EXISTS idx_tasks_category_id
  ON tasks(category_id);

```

Figure D.1: Complete PostgreSQL Schema Definition (server/sql/schema.sql)

Appendix E: Glossary of Technical Terms

The following glossary defines key technical terms used throughout this thesis document.

Term	Definition
Agile	An iterative and incremental software development methodology that values working software, collaboration, and responsiveness to change over comprehensive planning and documentation.
API	Application Programming Interface — a defined set of rules and endpoints enabling software components to communicate with each other.
bcrypt	A password hashing function designed to be computationally expensive and resistant to brute-force and dictionary attacks through the use of a configurable work factor (salt rounds).
CORS	Cross-Origin Resource Sharing — a browser security mechanism that controls which web origins are permitted to make cross-origin HTTP requests to a server.
CRUD	Create, Read, Update, Delete — the four fundamental operations performed on persistent data in a database-backed application.
CSS Custom Properties	Variables defined in CSS using the --variable-name syntax that allow reusable design values (colors, spacing, fonts) to be defined once and referenced throughout a stylesheet.
DOM	Document Object Model — a programming interface representing an HTML document as a tree of objects that can be manipulated by JavaScript.
ERD	Entity-Relationship Diagram — a visual representation of the entities (tables) in a database and the relationships and constraints between them.
Express.js	A minimal and flexible Node.js web application framework providing routing, middleware integration, and HTTP request/response handling utilities.
Fetch API	A modern JavaScript browser API for making asynchronous HTTP requests, replacing the older XMLHttpRequest interface.
Foreign Key	A database column that references the primary key of another table, enforcing referential integrity between related records.
Full-Stack	A development approach that encompasses both the frontend (client-side presentation) and backend (server-side logic and database) components of a web application.
Node.js	A server-side JavaScript runtime built on Google's V8 JavaScript engine, providing non-blocking, event-driven I/O capabilities for scalable web server development.
PostgreSQL	An open-source, ACID-compliant object-relational database management system known for its robustness, standards compliance, and extensibility.

Term	Definition
PWA	Progressive Web App — a web application that uses modern browser APIs (service workers, Web App Manifest) to deliver native app-like capabilities including offline support and home screen installation.
Rate Limiting	A technique that restricts the number of requests a client can make to an API endpoint within a defined time window, protecting against brute-force and denial-of-service attacks.
REST / RESTful	Representational State Transfer — an architectural style for designing networked APIs using HTTP methods (GET, POST, PUT, PATCH, DELETE) and resource-oriented URLs.
Session	A server-side mechanism for maintaining user state across multiple stateless HTTP requests, identified by a session ID stored in a browser cookie.
Scrum	A specific Agile framework organizing development into fixed-length sprints with defined roles (Product Owner, Scrum Master, Development Team) and ceremonies (planning, review, retrospective).
Sprint	A time-boxed, fixed-length development iteration in the Scrum/Agile framework (typically 2–6 weeks) that produces a potentially shippable software increment.
SQL Injection	A web security attack where malicious SQL code is inserted into input fields with the intention of manipulating database queries. Prevented through parameterized queries.
UUID	Universally Unique Identifier — a 128-bit identifier generated to be globally unique, used as primary keys in the Task Flow database to prevent sequential ID enumeration.
XSS	Cross-Site Scripting — a web security vulnerability where malicious client-side scripts are injected into web pages viewed by other users. Prevented through HTML entity escaping.

